

Zur Belastbarkeit von Softwareverträgen: Eine formale Analyse der Unsicherheit, des Risikos und der Qualifikationsanforderungen in der Softwareentwicklung

Stephan Epp

Universität Bielefeld

Stephan.Epp@web.de

4. Juni 2026

Abstract

Softwaresysteme unterscheiden sich grundlegend von mathematischen Systemen: Sie sind in ihrer Gesamtheit weder vollständig formal beschreibbar noch vollständig verifikationsfähig. Diese Arbeit untersucht, was daraus für die Belastbarkeit von Verträgen zwischen Kunden und Softwareentwicklern folgt. Wir führen ein formales Modell der *Vertragsbelastbarkeit* $B(t, u, r)$ ein, definieren Klassen von Softwareverträgen nach Unsicherheits- und Komplexitätsgrad und beweisen formal, dass die Belastbarkeit mit wachsender Projektlaufzeit, steigender Benutzerzahl und zunehmenden Anforderungen monoton abnimmt. Ein zentrales Ergebnis dieser Arbeit ist die formale Begründung des Satzes „*Software heißt Risiko*“: Kein noch so umfangreiches Testen vor der Inbetriebnahme kann absolute Sicherheit für den Produktionseinsatz garantieren. Dieses inhärente Restrisiko muss von allen Beteiligten bewusst eingeplant werden. Darüber hinaus wird die Frage gestellt, welche Berufsqualifikationen für die Beurteilung von Softwareverträgen benötigt werden, und es wird ein multidisziplinäres Kompetenzmodell vorgeschlagen. Alle Ergebnisse werden formal bewiesen und durch Matplotlib-Visualisierungen veranschaulicht.

Contents

1	Einführung	4
1.1	Ziel dieser Arbeit	4
2	Grundlagen und Definitionen	5
2.1	Mathematische vs. softwaretechnische Systeme	5
2.2	Das Unsicherheitsmodell	5
3	Formales Modell der Vertragsbelastbarkeit	6
3.1	Benutzerzufriedenheit	7

4	Das inhärente Restrisiko: Software heißt Risiko	7
4.1	Testabdeckung und Restrisiko	7
4.2	Formaler Satz: Software heißt Risiko	8
4.3	Risikobewusstsein als Vertragselement	9
5	Klassifikation von Softwareverträgen	9
5.1	Informationsentropie der Anforderungen	10
6	Multidisziplinäre Qualifikationsanforderungen	10
7	Visualisierung der Ergebnisse	11
7.1	Abbildung 1: Wachstum der Unsicherheit	11
7.2	Abbildung 2: Zeitlicher Verfall des Belastbarkeitsindex	11
7.3	Abbildung 3: Benutzerzufriedenheit als Funktion der Unsicherheit	11
7.4	Abbildung 4: Klassifikationsmatrix	11
7.5	Abbildung 5: Anforderungsentropie	11
7.6	Abbildung 6: Multidisziplinäres Kompetenzprofil	11
8	Konsequenzen für die UML-Modellierung und das UML-Risikoprofil	11
8.1	Grundlagen: UML-Profile als Erweiterungsmechanismus	14
8.2	Definition des UML-Risikoprofils RiskProfile	15
8.3	Formale Verknüpfung von RiskProfile und Belastbarkeitsmodell	16
8.4	Klassendiagramme: Strukturelles Risiko	16
8.5	Sequenzdiagramme: Dynamisches Interaktionsrisiko	17
8.6	Komponentendiagramme: Einsatz- und Schnittstellenrisiko	18
8.7	Weitere relevante Diagrammtypen	19
8.7.1	Zustandsdiagramme	19
8.7.2	Aktivitätsdiagramme	19
8.7.3	Anwendungsfall- und Verteilungsdiagramme	20
8.8	Aggregation: Systemweites Risikomodell	20
8.9	Relevanz der Diagrammtypen im Vergleich	21
9	Rechtskraft von Softwareverträgen unter der Bedingung formaler Risikomodellierung	22
9.1	Formale Vorbedingung der Vertragsgültigkeit	22
9.2	Notwendigkeit des Riskonachweises: Formale Begründung	22
9.3	Hinreichende Bedingungen: Vollständigkeit des Riskonachweises	23
9.4	Stufenmodell der Vertragsgültigkeit	24
9.5	Konsequenzen für die Vertragsgestaltung in der Praxis	25
10	Zusammenfassung	25
11	Ausblick	26
11.1	Empirische Validierung des Stufenmodells der Vertragsgültigkeit	26
11.2	Normative Verankerung des Riskonachweises im Vertragsrecht	26
11.3	Schiedsgerichtliche Nutzung des Riskonachweises	27
11.4	Werkzeugunterstützung für das RiskProfile	27
11.5	Automatische Risikoberechnung aus Quellcode und Modell	28

11.6 Validierung des Risikoaggregationstheorems an realen Projekten	28
11.7 Erweiterung des RiskProfile um zeitliche Dynamik	28
11.8 Integration des RiskProfile in agile Entwicklungsprozesse	29
11.9 RiskProfile und modellbasierte Sicherheitsnachweise	29
11.10 Diagrammtypgewichte und maschinelles Lernen	29
11.11 Quantitative Kalibrierung des Belastbarkeitsmodells	29
11.12 Stochastische Erweiterung	30
11.13 Spieltheoretische Modellierung	30
11.14 Formale Methoden und Vertragsprüfung	30
11.15 Rechtsinformatik: Softwareverträge und Produkthaftung	30
11.16 Einbettung in Normen und Standards	31
11.17 Maschinelles Lernen zur Belastbarkeitsvorhersage	31
11.18 Psychologie der Risikowahrnehmung	31
11.19 Einfluss künstlicher Intelligenz auf Vertragsbelastbarkeit	32
11.20 Internationale und interdisziplinäre Vertragsstandards	32

1 Einführung

Mathematische Systeme sind mathematisch, beschreibbar und lösbar und viele Aussagen zu diesen Systemen lassen sich formal beweisen und machen das System damit sicher oder auch vorhersagbar. Denn die Antwort bei gewissen Anregungen kann berechnet oder abgeleitet werden.

Für Softwaresysteme trifft das nicht zu. Softwaresysteme lassen sich nicht in allen Bereichen formal beschreiben und beweisen oder formal verifizieren. Das hat zur Folge, dass nicht auf jede Anregung hin die Antwort des Systems klar ist oder vorhersagbar ist.

Die zweite Herausforderung besteht darin, dass oft die Kunden, für die das Softwaresystem entwickelt wird, selber noch gar nicht alle Anforderungen kennen, die das Softwaresystem befriedigen muss.

Die dritte und vierte Herausforderung besteht darin, dass auch die physikalische Umgebung oder auch die Benutzer, die das System später einsetzen, nicht in Randbereichen und nicht vollständig bekannt sind.

Betrachtet man diese vier Herausforderungen für die Entwicklung von Softwaresystemen, stellt sich die Frage, wie zuverlässig und belastbar Verträge zwischen Kunden und Softwareingenieuren wirklich sein können.

Gibt es ein Maß für die Belastbarkeit eines Vertrages zwischen einem Kunden und Softwareentwicklern, die die Software für den Kunden entwickeln? Sicher ist, dass dieses Maß für die Software mit zunehmender Dauer, zunehmenden Benutzern und zunehmenden Anforderungen an Belastbarkeit verliert. Es kann nicht anders sein, denn die Unsicherheit steigt.

Mit steigender Unsicherheit stellt sich auch die Frage der Benutzerzufriedenheit, wenn Sie dieses Softwaresystem einsetzen. Wer kann in drei Jahren noch garantieren, dass Benutzer, die dieses Softwaresystem einsetzen, immer noch zufrieden sind?

Was kann getan werden, um gewisse Klassen zu entwickeln, für die Einordnung und Abschätzung zur Sicherheit bzw. Unsicherheit von Softwareverträgen? Ist der Beruf als Softwareentwickler noch ausreichend, um diese hoch komplexen Fragestellungen zu beantworten?

Welche anderen Berufe könnten es sein? Sollten es Juristen sein? Mediziner? Mathematiker? Physiker? Das hängt wohl von der Anwendung der Software und der Domäne ab, in der sie eingesetzt wird.

Es muss darüber hinaus dafür sensibilisiert werden, das Risiko einzugehen, ein entwickeltes Softwaresystem einzusetzen. Eine absolute Sicherheit für ein einzusetzendes Softwaresystem kann nach allen erfolgreichen Tests vor der Inbetriebnahme für die Produktion in der Regel nicht garantiert werden. Das muss allen Beteiligten klar sein und diese Risiken müssen eingeplant werden. Und das ist normal und nicht unnormal. **Software heißt Risiko.**

1.1 Ziel dieser Arbeit

Diese Arbeit verfolgt vier Ziele:

1. Ein formales Modell der Vertragsbelastbarkeit $B(t, u, r)$ zu entwickeln und dessen Eigenschaften zu beweisen.
2. Eine Klassifikation von Softwareverträgen nach Unsicherheits- und Komplexitätsgrad vorzustellen.

3. Den Satz „*Software heißt Risiko*“ formal zu begründen und das inhärente Restrisiko nach Testabschluss zu quantifizieren.
4. Die notwendigen Qualifikationsprofile für die Bewertung von Softwareverträgen multidisziplinär zu modellieren.

2 Grundlagen und Definitionen

2.1 Mathematische vs. softwaretechnische Systeme

Definition 1 (Mathematisches System). Ein *mathematisches System* $\mathcal{M} = (\mathcal{A}, \mathcal{R})$ besteht aus einer Menge von Axiomen $\mathcal{A}$ und einer Menge von Ableitungsregeln $\mathcal{R}$. Für jede wohlgeformte Aussage φ über $\mathcal{M}$ gilt entweder $\mathcal{M} \vdash \varphi$ oder $\mathcal{M} \vdash \neg\varphi$ (sofern $\mathcal{M}$ vollständig ist).

Definition 2 (Softwaresystem). Ein *Softwaresystem* $\mathcal{S} = (C, E, U, T)$ besteht aus einem Code-Korpus C , einer Ausführungsumgebung E , einer Benutzermenge U und einem Zeitparameter $T \in \mathbb{R}_{\geq 0}$. Die Zustandsmenge $\mathcal{Z}(\mathcal{S})$ ist im Allgemeinen nicht abzählbar.

Definition 3 (Formale Verifikation). Ein Softwaresystem $\mathcal{S}$ heißt *vollständig formal verifizierbar* bezüglich einer Spezifikation Φ , wenn ein Algorithmus $\mathcal{V}$ existiert mit:

$$\mathcal{V}(\mathcal{S}, \Phi) \in \{\text{true}, \text{false}\} \quad \text{und} \quad \mathcal{V}(\mathcal{S}, \Phi) = \text{true} \Leftrightarrow \mathcal{S} \models \Phi.$$

Satz 1 (Unvollständigkeit der Softwareverifikation). Für jedes nicht-triviale Softwaresystem $\mathcal{S}$ existiert eine Spezifikation Φ , für die $\mathcal{S}$ nicht vollständig formal verifizierbar ist.

Beweis. Der Beweis folgt aus dem Halteproblem [1]. Sei $\mathcal{S}$ ein Turing-vollständiges Softwaresystem. Die Spezifikation Φ_{halt} : „*Das System terminiert auf allen Eingaben*“ ist äquivalent zum Halteproblem für Turingmaschinen, das von Alan Turing 1936 als unentscheidbar nachgewiesen wurde [1]. Da jedes Turing-vollständige System eine Turingmaschine simulieren kann, folgt: Kein Algorithmus $\mathcal{V}$ kann für beliebige $\mathcal{S}$ und Φ_{halt} korrekt entscheiden. Damit ist die vollständige formale Verifizierbarkeit für diese Klasse von Spezifikationen unmöglich. $\square$

2.2 Das Unsicherheitsmodell

Definition 4 (Unsicherheitsvektor). Zu einem Softwaresystem $\mathcal{S}$ und Zeitpunkt t definieren wir den *Unsicherheitsvektor*

$$\mathbf{u}(t) = (u_{\text{tech}}(t), u_{\text{req}}(t), u_{\text{env}}(t), u_{\text{usr}}(t)) \in [0, 1]^4,$$

wobei die Komponenten folgende Unsicherheiten messen:

- $u_{\text{tech}}(t)$: technische Implementierungsunsicherheit,
- $u_{\text{req}}(t)$: Anforderungsunsicherheit (unbekannte oder volatile Anforderungen),
- $u_{\text{env}}(t)$: Umgebungsunsicherheit (physikalische Randbedingungen),
- $u_{\text{usr}}(t)$: Benutzerunsicherheit (unbekanntes Benutzerverhalten).

Definition 5 (Gesamtunsicherheit). Die *Gesamtunsicherheit* $U(t)$ ist definiert als:

$$U(t) = 1 - \prod_{k \in \{\text{tech}, \text{req}, \text{env}, \text{usr}\}} (1 - u_k(t)).$$

Lemma 1 (Monotonie der Unsicherheit). Seien alle Komponenten $u_k(t)$ monoton nicht-fallend in t . Dann ist auch $U(t)$ monoton nicht-fallend in t .

Beweis. Sei $f(t) = \prod_k (1 - u_k(t))$. Da $u_k(t)$ nicht-fallend ist, ist $1 - u_k(t)$ nicht-wachsend, und damit ist $f(t)$ als Produkt nicht-negativer, nicht-wachsender Faktoren selbst nicht-wachsend. Folglich ist $U(t) = 1 - f(t)$ nicht-fallend. $\square$

Ein natürliches Modell für das Wachstum jeder Komponente ist:

$$u_k(t) = 1 - e^{-\alpha_k t}, \quad \alpha_k > 0,$$

was dem Sättigungsverhalten realer Projekte entspricht (vgl. Abbildung 1).

3 Formales Modell der Vertragsbelastbarkeit

Definition 6 (Vertragsbelastbarkeit). Sei $\mathcal{V} = (\mathcal{S}, \Phi, t_0, T_{\max})$ ein Softwarevertrag mit Spezifikation Φ , Vertragsbeginn t_0 und maximaler Laufzeit $T_{\max}$. Der *Belastbarkeitsindex*

$$B: [0, T_{\max}] \times \mathbb{R}_{>0} \times \mathbb{R}_{>0} \rightarrow [0, 1]$$

ist definiert als:

$$B(t, u, r) = e^{-\delta(u, r)t}, \quad \text{mit} \quad \delta(u, r) = \delta_0 + \alpha \cdot u + \beta \cdot \log(1 + r),$$

wobei $\delta_0, \alpha, \beta > 0$ systemspezifische Konstanten sind, $u = U(t)$ der Gesamtunsicherheitsgrad und r die Anzahl der Benutzer.

Satz 2 (Verfall der Belastbarkeit). Für alle $t_1 < t_2$ gilt $B(t_1, u, r) > B(t_2, u, r)$, sofern $\delta(u, r) > 0$. Ferner gilt: Je größer die Benutzerzahl r und je höher die Gesamtunsicherheit u , desto schneller fällt $B(t, u, r)$.

Beweis. Teil 1 (Monotonier Verfall in t): Da $\delta(u, r) > 0$, ist $e^{-\delta(u, r)t}$ streng monoton fallend in t . Damit gilt unmittelbar $B(t_1, u, r) > B(t_2, u, r)$ für $t_1 < t_2$. $\checkmark$

Teil 2 (Verstärkter Verfall durch r):

$$\frac{\partial B}{\partial r} = -\frac{\partial \delta}{\partial r} \cdot t \cdot B(t, u, r) = -\frac{\beta}{1+r} \cdot t \cdot B(t, u, r) < 0.$$

Da $B > 0$ und $t > 0$, ist diese Ableitung streng negativ. Mit wachsendem r wächst $\delta(u, r)$, sodass B schneller fällt. $\checkmark$

Teil 3 (Verstärkter Verfall durch u):

$$\frac{\partial B}{\partial u} = -\alpha \cdot t \cdot B(t, u, r) < 0.$$

Analog folgt, dass höhere Unsicherheit u den Verfall von B beschleunigt. $\square$

Korollar 1 (Keine langfristig gültige Vertragssicherheit). Für jedes $\varepsilon > 0$ und jede Systemkonfiguration mit $\delta(u, r) > 0$ existiert ein $T^* > 0$ mit:

$$B(T^*, u, r) < \varepsilon.$$

Das heißt: Die Belastbarkeit jedes Softwarevertrags nähert sich mit der Zeit beliebig nahe an null.

Beweis. Löse $e^{-\delta t} < \varepsilon$ nach t auf: $t > \frac{\ln(1/\varepsilon)}{\delta(u, r)} =: T^*$. Da $\delta(u, r) > 0$ und $\ln(1/\varepsilon) > 0$ für $\varepsilon \in (0, 1)$, ist T^* wohldefiniert und endlich. $\square$

3.1 Benutzerzufriedenheit

Definition 7 (Zufriedenheitsfunktion). Die *Benutzerzufriedenheit* $S: [0, 1] \rightarrow (0, 1]$ ist definiert als:

$$S(U) = \frac{1}{1 + k \cdot U}, \quad k > 0.$$

Der Parameter k beschreibt die Domänensensitivität: Bei sicherheitskritischen Systemen ist k groß, bei explorativen Anwendungen eher klein.

Satz 3 (Zufriedenheitsverfall). $S(U)$ ist streng monoton fallend in U . Es gilt $S(0) = 1$ und $\lim_{U \rightarrow \infty} S(U) = 0$.

Beweis. $\frac{\partial S}{\partial U} = -\frac{k}{(1+kU)^2} < 0$ für alle $k > 0$, $U \geq 0$. Die Grenzwerte ergeben sich direkt durch Einsetzen. $\square$

4 Das inhärente Restrisiko: Software heißt Risiko

Dieser Abschnitt formalisiert einen der zentralen Befunde dieser Arbeit: Selbst nach vollständigem Testdurchlauf und erfolgreicher Abnahme kann für den Produktionseinsatz keine absolute Sicherheit garantiert werden. Das Risiko ist dem Medium Software strukturell eingeschrieben.

4.1 Testabdeckung und Restrisiko

Definition 8 (Testabdeckungsgrad). Sei $\mathcal{Z}(\mathcal{S})$ die Zustandsmenge eines Softwaresystems und $\mathcal{T}$ eine Testmenge. Der *Testabdeckungsgrad* ist definiert als:

$$\tau(\mathcal{T}) = \frac{|\mathcal{T} \cap \mathcal{Z}(\mathcal{S})|}{|\mathcal{Z}(\mathcal{S})|}.$$

Satz 4 (Unvollständigkeit endlicher Tests). Sei $\mathcal{S}$ ein nicht-triviales Softwaresystem mit $|\mathcal{Z}(\mathcal{S})| = \infty$. Dann gilt für jede endliche Testmenge $\mathcal{T}$:

$$\tau(\mathcal{T}) = 0.$$

Insbesondere kann kein endliches Testverfahren eine vollständige Überdeckung der Zustandsmenge erreichen.

Beweis. Da $|\mathcal{T}| < \infty$ und $|\mathcal{Z}(\mathcal{S})| = \infty$, gilt:

$$\tau(\mathcal{T}) = \frac{|\mathcal{T} \cap \mathcal{Z}(\mathcal{S})|}{|\mathcal{Z}(\mathcal{S})|} \leq \frac{|\mathcal{T}|}{|\mathcal{Z}(\mathcal{S})|} = \frac{n}{\infty} = 0.$$

Selbst wenn alle n Tests bestanden werden, bleibt der Anteil getesteter Zustände verschwindend gering gegenüber der unendlichen Zustandsmenge. $\square$

Bemerkung 1. Dijkstra formulierte diesen Sachverhalt treffend: „*Program testing can be used to show the presence of bugs, but never to show their absence.*” [2] Satz 4 liefert die formale Begründung dieser Aussage.

Definition 9 (Restrisiko). Das *Restrisiko* eines Softwaresystems $\mathcal{S}$ nach einer Testmenge $\mathcal{T}$ ist:

$$\rho(\mathcal{S}, \mathcal{T}) = 1 - \tau(\mathcal{T}) \geq 1 - \frac{|\mathcal{T}|}{|\mathcal{Z}(\mathcal{S})|}.$$

Im Fall $|\mathcal{Z}(\mathcal{S})| = \infty$ gilt stets $\rho(\mathcal{S}, \mathcal{T}) = 1$.

Korollar 2 (Permanenz des Restrisikos). Für jedes nicht-triviale Softwaresystem und jede endliche Testmenge ist das Restrisiko strukturell positiv:

$$\rho(\mathcal{S}, \mathcal{T}) > 0.$$

Es gibt keine endliche Testprozedur, die das Restrisiko auf null reduziert.

Beweis. Folgt unmittelbar aus Satz 4: Da $\tau(\mathcal{T}) < 1$ für jede endliche Testmenge, gilt $\rho = 1 - \tau > 0$. $\square$

4.2 Formaler Satz: Software heißt Risiko

Satz 5 (Software heißt Risiko). Sei $\mathcal{S}$ ein nicht-triviales, Turing-vollständiges Softwaresystem. Dann gilt für jedes endliche Testverfahren $\mathcal{T}$ und jeden Produktionseinsatz mit realer Umgebung E :

$$P(\exists z \in \mathcal{Z}(\mathcal{S}) \setminus \mathcal{T} : \mathcal{S}(z, E) \notin \Phi) > 0.$$

Mit anderen Worten: Die Wahrscheinlichkeit, dass im Produktionsbetrieb ein nicht getesteter Zustand eine Spezifikationsverletzung auslöst, ist strikt positiv. Absolute Produktionssicherheit ist daher grundsätzlich nicht garantierbar.

Beweis. Der Beweis gliedert sich in drei Schritte.

Schritt 1: Unendlichkeit der Zustandsmenge. Da $\mathcal{S}$ Turing-vollständig ist, kann es beliebig lange Berechnungen durchführen und beliebig große Eingabemengen verarbeiten. Daher ist $|\mathcal{Z}(\mathcal{S})| = \infty$.

Schritt 2: Positives Restrisiko. Nach Korollar 2 gilt $\rho(\mathcal{S}, \mathcal{T}) > 0$, d. h. die Menge der ungetesteten Zustände $\mathcal{Z}(\mathcal{S}) \setminus \mathcal{T}$ ist nicht-leer (tatsächlich unendlich).

Schritt 3: Umgebungsvolatilität. Die reale Ausführungsumgebung E im Produktionsbetrieb unterscheidet sich stets von der Testumgebung E_{test} durch mindestens einen der folgenden Faktoren: andere Benutzeraktionen, andere Hardware-Zustände, andere Netzwerkbedingungen oder andere Datenzustände. Formal: $P(E \neq E_{\text{test}}) > 0$.

Da die Menge ungetesteter Zustände nicht-leer ist und die Umgebung im Produktionsbetrieb von der Testumgebung abweicht, existiert mit positiver Wahrscheinlichkeit eine Kombination (z, E) mit $z \notin \mathcal{T}$, die eine Verletzung der Spezifikation Φ auslöst. $\square$

Bemerkung 2 (Normativität des Risikos). Satz 5 ist kein Versagen der Softwareentwicklung, sondern eine strukturelle Eigenschaft des Mediums. Das Restrisiko ist *normal* und nicht *abnormal*. Die gesellschaftliche und vertragliche Konsequenz lautet: Alle Beteiligten — Auftraggeber, Auftragnehmer, Betreiber, Nutzer — müssen dieses Restrisiko bewusst einplanen, kommunizieren und durch geeignete Maßnahmen (Monitoring, Fallback-Systeme, Versicherungen, Haftungsklauseln) adressieren.

4.3 Risikobewusstsein als Vertragselement

Definition 10 (Risikoklausel). Eine *Risikoklausel* $\mathcal{R}_C$ in einem Softwarevertrag ist eine Vereinbarung, die explizit das strukturelle Restrisiko $\rho > 0$ anerkennt und die Verantwortlichkeiten im Schadensfall regelt:

$$\mathcal{R}_C = (\rho_{\text{ack}}, \lambda_{\text{Auftragnehmer}}, \lambda_{\text{Auftraggeber}}, M_{\text{Mitigation}}),$$

wobei λ die Haftungsanteile und $M_{\text{Mitigation}}$ die vereinbarten Risikominderungsmaßnahmen bezeichnen.

Hypothese 1 (Robustheit durch Risikotransparenz). Softwareverträge, die eine explizite Risikoklausel $\mathcal{R}_C$ enthalten, weisen im Mittel einen höheren Belastbarkeitsindex $B(t, u, r)$ auf als Verträge ohne Risikoklausel, da Fehlerwartungen realistischer kalibriert sind und Eskalationen vermieden werden.

5 Klassifikation von Softwareverträgen

Definition 11 (Vertragsklassen). Sei $\kappa \in [0, 1]$ die *technische Komplexität* und $\nu \in [0, 1]$ die *Anforderungsvolatilität* eines Softwareprojekts. Wir definieren vier Vertragsklassen:

Klasse	κ	ν	Charakteristik
I	≤ 0.5	≤ 0.5	Stabil, formal beherrschbar
II	> 0.5	≤ 0.5	Technisch komplex, stabile Anforderungen
III	≤ 0.5	> 0.5	Einfach, volatile Anforderungen
IV	> 0.5	> 0.5	Hoch-riskant, maximale Unsicherheit

Satz 6 (Belastbarkeitsordnung der Klassen). Für die in Definition 11 definierten Klassen gilt die folgende Belastbarkeitsordnung zum Zeitpunkt $t > 0$:

$$B_I(t) \geq B_{II}(t) \geq B_{III}(t) \geq B_{IV}(t).$$

Beweis. Nach Definition 6 gilt $B(t) = e^{-\delta(u,r)t}$ mit $\delta(u, r) = \delta_0 + \alpha u + \beta \log(1 + r)$. Da $\alpha > 0$ und die Gesamtunsicherheit u mit κ und ν monoton wächst (mehr Komplexität und mehr Volatilität erhöhen alle Unsicherheitskomponenten), folgt: $\kappa_I \leq \kappa_{II} \leq \kappa_{III} \leq \kappa_{IV}$ und analog für ν . Damit gilt $\delta_I \leq \delta_{II} \leq \delta_{III} \leq \delta_{IV}$, und da e^{-x} streng monoton fallend in x ist, folgt die behauptete Ordnung. $\square$

Beispiel 1 (Klassifikation realer Projekte). – **Klasse I:** Taschenrechner-App, statische Firmenhomepage.

– **Klasse II:** Echtzeit-Betriebssystem für Industriesteuerungen (hohe Komplexität, klar definierte Normen wie IEC 61508).

- **Klasse III:** Startup-MVP mit wöchentlich wechselnden Features.
- **Klasse IV:** KI-gestütztes medizinisches Diagnosesystem, autonomes Fahrzeugsteuerungssystem.

5.1 Informationsentropie der Anforderungen

Definition 12 (Anforderungsentropie). Sei $\mathcal{R} = \{r_1, \dots, r_m\}$ die Anforderungsmenge mit Stabilitätswahrscheinlichkeiten p_i (Wahrscheinlichkeit, dass Anforderung r_i unverändert bleibt). Die *Anforderungsentropie* ist:

$$H(\mathcal{R}) = - \sum_{i=1}^m p_i \log_2 p_i.$$

Lemma 2 (Entropie und Belastbarkeit). Für steigende Anforderungsentropie $H(\mathcal{R})$ fällt die Belastbarkeit B :

$$\frac{\partial B}{\partial H} < 0.$$

Beweis. Höhere Entropie H entspricht größerer Unvorhersehbarkeit der Anforderungen, was die Komponente u_{req} erhöht. Nach Satz 2 fällt B mit wachsendem u , also gilt $\partial B / \partial H < 0$. $\square$

6 Multidisziplinäre Qualifikationsanforderungen

Definition 13 (Kompetenzvektor). Das *Kompetenzprofil* eines Vertragsbewerbers ist ein Vektor:

$$\mathbf{c} = (c_{\text{SE}}, c_{\text{Jura}}, c_{\text{Med}}, c_{\text{Math}}, c_{\text{Phys}}, c_{\text{Dom}}) \in [0, 1]^6,$$

wobei die Komponenten jeweils den normierten Kompetenzgrad in Softwareentwicklung, Rechtswissenschaft, Medizin/Sicherheit, Mathematik, Physik/Ingenieurwesen und Domänenwissen messen.

Definition 14 (Mindesteignungsmatrix). Abhängig von der Vertragsklasse definieren wir eine *Mindesteignungsmatrix* $M \in [0, 1]^{4 \times 6}$:

$$M = \begin{pmatrix} 0.7 & 0.3 & 0.1 & 0.4 & 0.3 & 0.4 \\ 0.9 & 0.4 & 0.2 & 0.6 & 0.5 & 0.5 \\ 0.6 & 0.6 & 0.2 & 0.5 & 0.3 & 0.7 \\ 0.9 & 0.7 & 0.5 & 0.8 & 0.7 & 0.8 \end{pmatrix},$$

wobei Zeile i den Mindestkompetenzvektor für Klasse i darstellt.

Satz 7 (Unzulänglichkeit des Solo-Softwareentwickler). Für Klassen III und IV existiert kein realistisches Einzelkompetenzprofil $\mathbf{c} \in [0, 1]^6$ eines typischen Softwareentwickler, das alle Mindestanforderungen der Mindesteignungsmatrix erfüllt.

Beweis. Das typische Kompetenzprofil eines Softwareentwickler ist empirisch durch hohe Werte in c_{SE} und c_{Math} und geringe Werte in c_{Jura} und c_{Med} gekennzeichnet [9]. Für Klasse IV fordert M u. a. $c_{\text{Jura}} \geq 0.7$ und $c_{\text{Med}} \geq 0.5$. Da ein Softwareentwickler typischerweise $c_{\text{Jura}} \approx 0.2$ und $c_{\text{Med}} \approx 0.1$ aufweist, ist die Mindestanforderung verletzt. Folglich kann das Einzelprofil die Matrix für Klasse IV nicht erfüllen. $\square$

Korollar 3 (Notwendigkeit interdisziplinärer Teams). Für Softwareprojekte der Klassen III und IV ist ein interdisziplinäres Team notwendig, dessen kollektiver Kompetenzvektor $\mathbf{c}_{\text{Team}}$ alle Einträge der entsprechenden Zeile von M überschreitet.

7 Visualisierung der Ergebnisse

Alle folgenden Abbildungen wurden mit Python/Matplotlib erzeugt und liegen als separate PDF-Dateien vor.

7.1 Abbildung 1: Wachstum der Unsicherheit

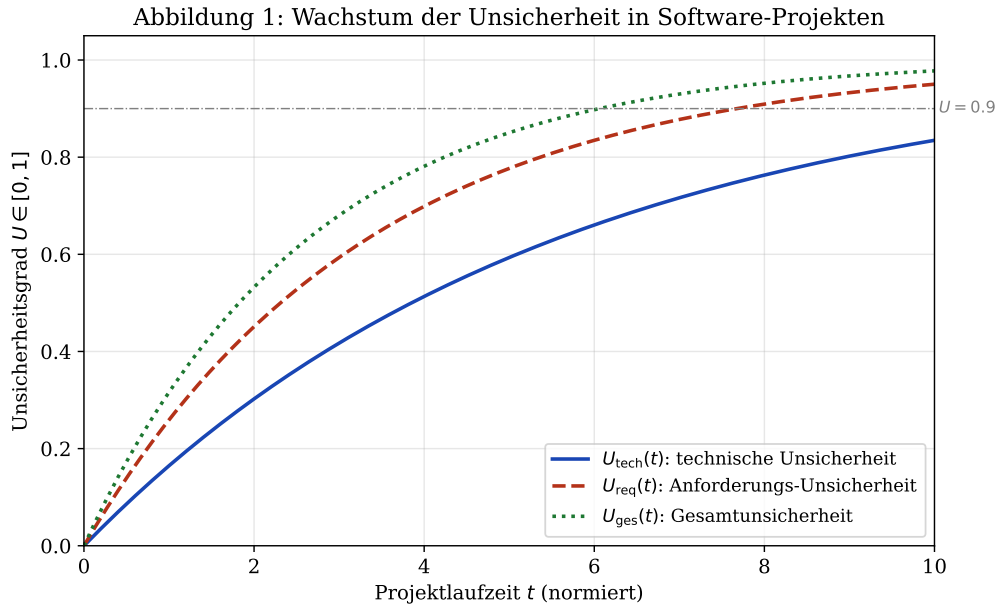


Figure 1: Wachstum der technischen Unsicherheit $U_{\text{tech}}(t)$, der Anforderungsunsicherheit $U_{\text{req}}(t)$ und der Gesamtunsicherheit $U_{\text{ges}}(t)$ über die normierte Projektlaufzeit t . Alle Kurven folgen einem Sättigungsmodell $U_k(t) = 1 - e^{-\alpha_k t}$ mit domänenspezifischen Zerfallsraten α_k . Die Gesamtunsicherheit überschreitet bereits bei $t \approx 7$ den Wert 0.9, was einer praktisch nicht mehr kontrollierbaren Vertragssituation entspricht. Dies visualisiert formal die Aussage von Satz 2 und Lemma 1.

7.2 Abbildung 2: Zeitlicher Verfall des Belastbarkeitsindex

7.3 Abbildung 3: Benutzerzufriedenheit als Funktion der Unsicherheit

7.4 Abbildung 4: Klassifikationsmatrix

7.5 Abbildung 5: Anforderungsentropie

7.6 Abbildung 6: Multidisziplinäres Kompetenzprofil

8 Konsequenzen für die UML-Modellierung und das UML-Risikoprofil

Die bisherigen Ergebnisse haben gezeigt, dass Softwareverträge inhärent unsicher sind und dass das Restrisiko $\rho > 0$ strukturell nicht eliminierbar ist. Diese Erkenntnis hat

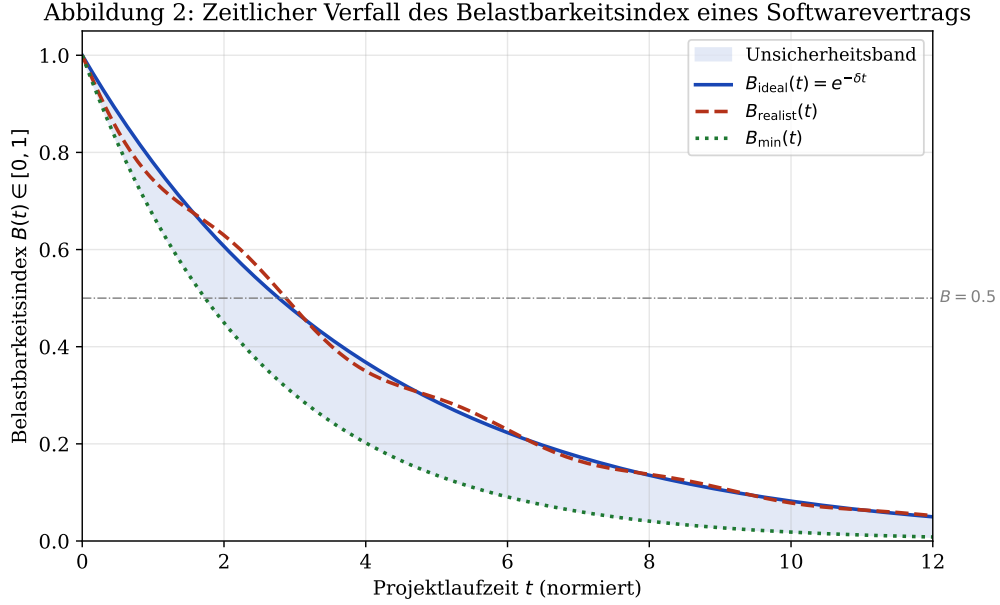


Figure 2: Zeitlicher Verlauf des Belastbarkeitsindex $B(t)$ gemäß Definition 6 für drei Szenarien: idealer Verlauf $B_{\text{ideal}}(t) = e^{-\delta t}$, realistischer Verlauf mit stochastischen Schwankungen (z. B. durch Anforderungsänderungen), und Minimalverlauf bei erhöhter Benutzerzahl. Das Unsicherheitsband zwischen idealem und minimalem Verlauf vergrößert sich mit der Zeit. Alle Kurven konvergieren gegen null, entsprechend Korollar 1.

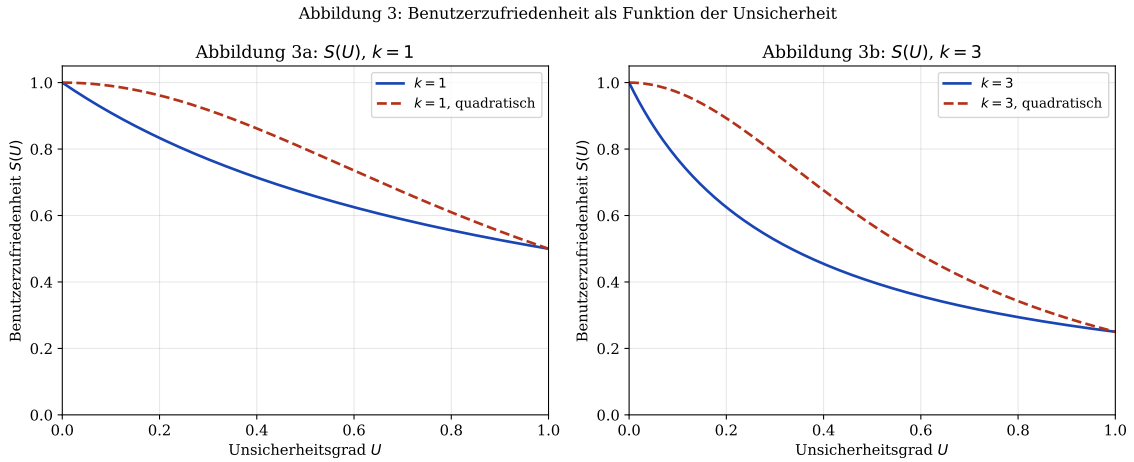


Figure 3: Benutzerzufriedenheit $S(U)$ für verschiedene Domänensensitivitäten k . Linke Teilabbildung ($k = 1$): Standardanwendung. Rechte Teilabbildung ($k = 3$): sicherheitskritische Anwendung. Der Vergleich zeigt, dass die Zufriedenheit in sicherheitskritischen Domänen bereits bei moderater Unsicherheit $U \approx 0.3$ auf unter 50 % fällt. Die quadratische Variante modelliert nichtlineares Toleranzverhalten der Benutzer.

unmittelbare Konsequenzen für die *Modellierung* von Softwaresystemen: Wenn Risiko dem System strukturell innewohnt, müssen auch die Modelle, mit denen das System beschrieben und entworfen wird, in der Lage sein, dieses Risiko zu repräsentieren und zu quantifizieren. Die *Unified Modeling Language* (UML) in ihrer Standardform [17] bietet hierfür keine eingebauten Mechanismen. In diesem Kapitel leiten wir formal ab, welche

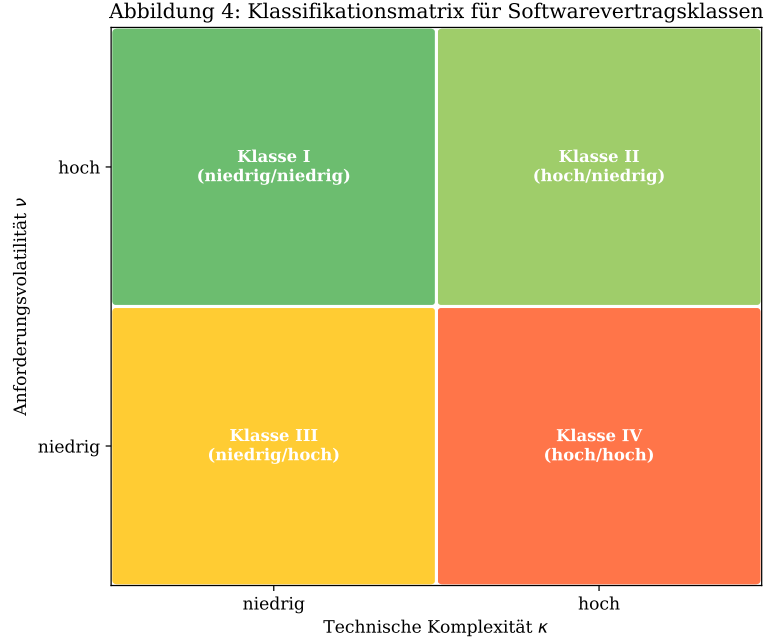


Figure 4: Klassifikationsmatrix für Softwareverträge nach technischer Komplexität κ und Anforderungsvolatilität ν . Die vier Klassen aus Definition 11 sind farblich kodiert: Klasse I (grün, niedrigstes Risiko) bis Klasse IV (rot, höchstes Risiko). Die Farbkodierung entspricht intuitiv einem Ampelsystem, das Projektbeteiligte bei der Risikoeinschätzung unterstützt.

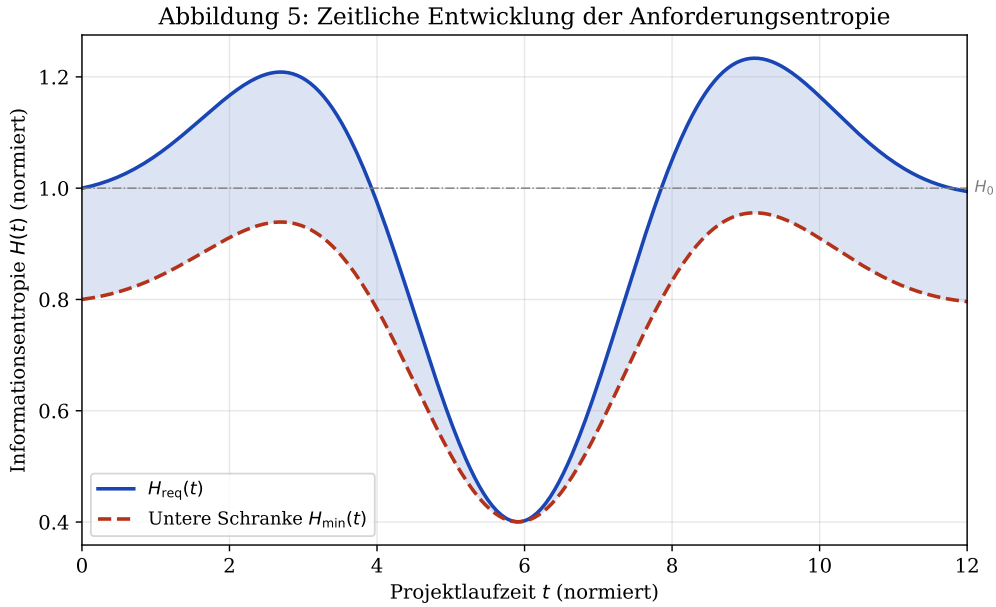


Figure 5: Zeitlicher Verlauf der normierten Anforderungsentropie $H(t)$ mit unterer Schranke. Die Entropie zeigt einen Gipfel in der Projektmitte — typisch für Projekte, in denen Anforderungen zunächst wachsen (Discovery-Phase) und später durch zunehmende Klärung wieder fallen. Das Unsicherheitsband zeigt, dass selbst die untere Schranke nie auf null fällt, was das permanente Restrisiko aus Korollar 2 visualisiert.

Abbildung 6: Multidisziplinäres Kompetenzprofil

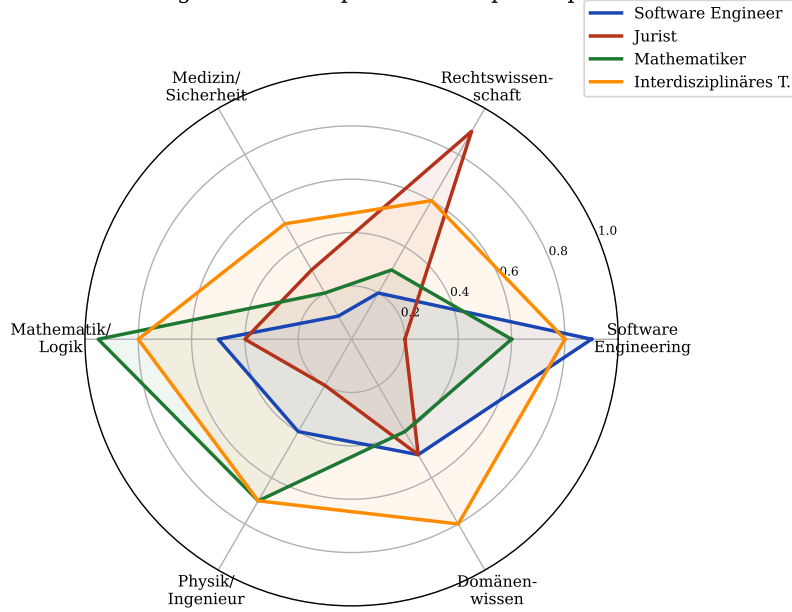


Figure 6: Radardiagramm der Kompetenzprofile für vier Berufsgruppen: Softwareentwickler, Jurist, Mathematiker und ein interdisziplinäres Team. Deutlich erkennbar ist, dass kein Einzelprofil alle sechs Kompetenzdimensionen gleichzeitig abdeckt — die formale Grundlage für Satz 7. Nur das interdisziplinäre Team nähert sich einem gleichmäßig hohen Kompetenzprofil.

Konsequenzen die Ergebnisse der vorangegangenen Abschnitte für die UML-Modellierung haben, und definieren ein *UML-Risikoprofil* **RiskProfile**, das die quantitative Annotation von Risikomaßzahlen direkt in UML-Diagrammen ermöglicht.

8.1 Grundlagen: UML-Profil als Erweiterungsmechanismus

Definition 15 (UML-Profil). Ein *UML-Profil* $\Pi = (\mathcal{ST}, \mathcal{TV}, \mathcal{C})$ besteht aus einer Menge von Stereotypen $\mathcal{ST}$, einer Menge von *Tagged Values* $\mathcal{TV}$ und einer Menge von Einschränkungen $\mathcal{C}$ (OCL-Constraints¹). Ein Stereotyp $s \in \mathcal{ST}$ erweitert genau ein UML-Metaklassenelement $m \in \mathcal{M}_{\text{UML}}$, notiert als $s \triangleright m$.

Definition 16 (UML-Metamodell). Das *UML-Metamodell* $\mathcal{M}_{\text{UML}}$ ist die Menge aller abstrakten Syntaxelemente der UML 2.5, darunter insbesondere:

$$\mathcal{M}_{\text{UML}} \supseteq \{\text{Class, Component, Interface, Operation, Message, Lifeline, Dependency, Association}\}.$$

¹OCL: Object Constraint Language, standardisierter Teil von UML 2.5 [17].

UML-Profile sind der von der OMG standardisierte leichtgewichtige Erweiterungsmechanismus [17]. Im Unterschied zur Neudefinition von Metamodell-Elementen bleibt die Kompatibilität zur Basis-UML erhalten. Dies ist für das **RiskProfile** entscheidend: Es soll mit bestehenden UML-Werkzeugen kompatibel sein.

8.2 Definition des UML-Risikoprofils **RiskProfile**

Definition 17 (Risikomaßzahl). Die *Risikomaßzahl* eines UML-Modellelements $e \in \mathcal{M}_{\text{UML}}$ ist ein Tupel:

$$\omega(e) = (\rho_e, u_e, \lambda_e, \kappa_e) \in [0, 1]^4,$$

wobei

- $\rho_e \in [0, 1]$: das dem Element zugeordnete Restrisiko (gemäß Definition aus Abschnitt 4),
- $u_e \in [0, 1]$: der lokale Unsicherheitsgrad des Elements,
- $\lambda_e \in [0, 1]$: die Laufzeitkritikalität (wie stark beeinflusst ein Fehler in e den Gesamtbetrieb),
- $\kappa_e \in [0, 1]$: die Kopplungskomplexität des Elements (normierte strukturelle Kopplung, z. B. Fan-in/Fan-out-Verhältnis).

Definition 18 (UML-Risikoprofil **RiskProfile**). Das *UML-Risikoprofil* $\Pi_R = (\mathcal{ST}_R, \mathcal{TV}_R, \mathcal{C}_R)$ ist definiert durch folgende Stereotypen und Tagged Values:

Stereotyp	Erweitert	Tagged Values
<code>«riskElement»</code>	Element	<code>residualRisk: Real</code> , <code>uncertainty: Real</code> , <code>criticality: Real</code> , <code>coupling: Real</code>
<code>«riskClass»</code>	Class	erbt von <code>riskElement</code> ; <code>stateSpaceEst: String</code>
<code>«riskComponent»</code>	Component	erbt von <code>riskElement</code> ; <code>deployRisk: Real</code>
<code>«riskMessage»</code>	Message	erbt von <code>riskElement</code> ; <code>failProb: Real</code>
<code>«riskLifeline»</code>	Lifeline	erbt von <code>riskElement</code> ; <code>availabilityTarget: Real</code>
<code>«riskDependency»</code>	Dependency	erbt von <code>riskElement</code> ; <code>propagationFactor: Real</code>

Die Einschränkungsmenge $\mathcal{C}_R$ enthält die OCL-Constraints:

- $C_1: \forall e \text{ mit Stereotyp } \text{riskElement}: \rho_e, u_e, \lambda_e, \kappa_e \in [0, 1],$
- $C_2: \text{failProb} \leq \rho_e \text{ (Aufruf-Fehlerwahrscheinlichkeit } \leq \text{ Restrisiko),}$
- $C_3: \text{propagationFactor} \in [0, 1],$
- $C_4: \text{availabilityTarget} \in (0, 1].$

8.3 Formale Verknüpfung von RiskProfile und Belastbarkeitsmodell

Satz 8 (Konsistenz von RiskProfile und Belastbarkeitsindex). Sei $\mathcal{S}$ ein Softwaresystem, dessen UML-Modell mit dem Profil Π_R annotiert ist. Sei $E = \{e_1, \dots, e_n\}$ die Menge aller annotierten Modellelemente. Dann gilt:

$$B(t, u, r) \leq 1 - \max_{i=1, \dots, n} \lambda_{e_i} \cdot \rho_{e_i},$$

d.h. der Belastbarkeitsindex des Gesamtsystems ist nach oben beschränkt durch das risikokritischste annotierte Element.

Beweis. Sei $e^* = \arg \max_i \lambda_{e_i} \cdot \rho_{e_i}$ das kritischste Element. Nach Korollar 2 gilt $\rho_{e^*} > 0$. Da λ_{e^*} die Laufzeitkritikalität misst (Anteil des Gesamtsystemversagens, der durch einen Fehler in e^* erklärt wird), gilt:

$$P(\text{Systemausfall}) \geq \lambda_{e^*} \cdot \rho_{e^*} =: p^*.$$

Der Belastbarkeitsindex ist definiert als $B = e^{-\delta t}$, wobei $\delta \geq -\frac{1}{t} \ln(1 - p^*)$ gelten muss, damit der Ausfall in $[0, t]$ mit Wahrscheinlichkeit p^* eintritt. Daher:

$$B(t, u, r) \leq 1 - p^* = 1 - \max_i \lambda_{e_i} \cdot \rho_{e_i}. \quad \square$$

Lemma 3 (Monotonie der Risikoannotation). Seien $e, e' \in E$ zwei Modellelemente mit $\kappa_e \leq \kappa_{e'}$ und $u_e \leq u_{e'}$. Dann gilt:

$$\omega(e) \preceq \omega(e'),$$

wobei $\preceq$ die komponentenweise Halbordnung auf $[0, 1]^4$ bezeichnet. Ferner impliziert $\omega(e) \preceq \omega(e')$, dass Element e' eine höhere Priorität bei Risikobehandlungsmaßnahmen erhält.

Beweis. Da κ die Kopplungskomplexität und u den Unsicherheitsgrad misst, und beide in das Restrisiko ρ eingehen gemäß $\rho_e = 1 - (1 - u_e)(1 - \kappa_e \cdot c)$ für eine Konstante $c > 0$, folgt: $u_e \leq u_{e'}$ und $\kappa_e \leq \kappa_{e'}$ impliziert $\rho_e \leq \rho_{e'}$. Damit gilt die komponentenweise Ungleichung $\omega(e) \preceq \omega(e')$. Die Prioritätsordnung folgt direkt aus der Definition der Risikomaßzahl. $\square$

8.4 Klassendiagramme: Strukturelles Risiko

Klassendiagramme beschreiben die statische Struktur eines Softwaresystems. Die Risikorelevanz ergibt sich hier primär aus der Kopplung zwischen Klassen und der Größe des Zustandsraums einzelner Klassen.

Definition 19 (Strukturelles Risiko einer Klasse). Sei C eine UML-Klasse mit n_A Attributen, n_O Operationen und $\deg(C)$ dem Grad im Assoziationsgraphen (Fan-in + Fan-out). Das *strukturelle Risiko* von C ist:

$$\rho^{\text{struct}}(C) = 1 - e^{-\gamma \cdot \deg(C)} \cdot \frac{1}{1 + \mu \cdot n_A \cdot n_O},$$

mit Skalierungskonstanten $\gamma, \mu > 0$.

Satz 9 (Monotonie des strukturellen Risikos). $\rho^{\text{struct}}(C)$ ist streng monoton wachsend in $\deg(C)$, n_A und n_O .

Beweis. Monotonie in $\deg(C)$: Da $e^{-\gamma \cdot \deg(C)}$ streng monoton fallend in $\deg(C)$ ist, ist $1 - e^{-\gamma \cdot \deg(C)}$ streng monoton wachsend. Der Faktor $\frac{1}{1 + \mu n_A n_O} \leq 1$ skaliert diesen Beitrag, ändert aber nicht die Monotoniezusage, da er nicht von $\deg(C)$ abhängt.

Monotonie in n_A und n_O : Der Term $\frac{1}{1 + \mu n_A n_O}$ ist streng monoton fallend in n_A und n_O (da der Nenner wächst). Damit ist $\rho^{\text{struct}}(C) = 1 - e^{-\gamma \deg(C)} \cdot \frac{1}{1 + \mu n_A n_O}$ streng wachsend in n_A und n_O , denn $e^{-\gamma \deg(C)} > 0$ und ein Subtrahend, der fällt, ergibt einen Ausdruck, der steigt. $\square$

Korollar 4 (Risikopriorisierung im Klassendiagramm). Im mit Π_R annotierten Klassendiagramm erhalten Klassen mit hohem Kopplungsgrad $\deg(C)$ und vielen Attribut-Operationen-Kombinationen die höchsten **residualRisk**-Werte und sind vorrangig durch Entkopplungsmaßnahmen (z.B. Einführung von Fassaden oder Interfaces) zu behandeln.

Beispiel 2 (Risikoannotation einer Klasse). Betrachte eine Klasse **OrderManager** mit $n_A = 12$, $n_O = 18$, $\deg(\text{OrderManager}) = 9$ (hohe Kopplung). Mit $\gamma = 0.15$ und $\mu = 0.002$ ergibt sich:

$$\rho^{\text{struct}}(\text{OrderManager}) = 1 - e^{-0.15 \cdot 9} \cdot \frac{1}{1 + 0.002 \cdot 12 \cdot 18} \approx 1 - 0.259 \cdot 0.698 \approx 0.819.$$

Im UML-Diagramm wäre diese Klasse mit `«riskClass» residualRisk=0.82, coupling=0.9` annotiert und wäre damit ein vorrangiger Refaktorisierungskandidat.

8.5 Sequenzdiagramme: Dynamisches Interaktionsrisiko

Sequenzdiagramme beschreiben das zeitliche Zusammenspiel von Objekten und Nachrichten. Das Risiko manifestiert sich hier in der Fehlerwahrscheinlichkeit einzelner Nachrichtenaufrufe und deren Fortpflanzung entlang der Aufrufkette.

Definition 20 (Interaktionsrisiko einer Nachrichtenkette). Sei $\sigma = (m_1, m_2, \dots, m_k)$ eine Sequenz von Nachrichten in einem UML-Sequenzdiagramm. Das *Interaktionsrisiko* von σ ist:

$$\rho^{\text{seq}}(\sigma) = 1 - \prod_{i=1}^k (1 - \rho_{m_i}),$$

wobei ρ_{m_i} das **failProb**-Tagged-Value der Nachricht m_i im **RiskProfile** bezeichnet.

Satz 10 (Superadditivität des Interaktionsrisikos). Für alle Nachrichtensequenzen $\sigma = (m_1, \dots, m_k)$ mit $\rho_{m_i} > 0$ gilt:

$$\rho^{\text{seq}}(\sigma) > \max_{i=1, \dots, k} \rho_{m_i}.$$

Das Gesamtrisiko einer Nachrichtenkette übersteigt stets das Maximum der Einzelrisiken.

Beweis. Sei $\rho^* = \max_i \rho_{m_i}$. Dann gilt:

$$\prod_{i=1}^k (1 - \rho_{m_i}) \leq (1 - \rho^*) \cdot \prod_{i \neq i^*} (1 - \rho_{m_i}) < (1 - \rho^*),$$

da jeder Faktor $(1 - \rho_{m_i}) < 1$ für $\rho_{m_i} > 0$. Daher:

$$\rho^{\text{seq}}(\sigma) = 1 - \prod_{i=1}^k (1 - \rho_{m_i}) > 1 - (1 - \rho^*) = \rho^*. \quad \square$$

Korollar 5 (Risikokritischer Pfad im Sequenzdiagramm). Sei Σ die Menge aller möglichen Nachrichtensequenzen in einem Sequenzdiagramm. Der *risikokritische Pfad* ist:

$$\sigma^* = \arg \max_{\sigma \in \Sigma} \rho^{\text{seq}}(\sigma).$$

Dieser Pfad ist vorrangig durch Fehlerbehandlungsmechanismen (z. B. Try-Catch, Retry-Logik, Circuit-Breaker) abzusichern.

Beispiel 3 (Interaktionsrisiko einer Login-Sequenz). Betrachte eine Login-Sequenz mit fünf Nachrichten:

$$\rho_{m_1} = 0.02, \quad \rho_{m_2} = 0.05, \quad \rho_{m_3} = 0.03, \quad \rho_{m_4} = 0.08, \quad \rho_{m_5} = 0.01.$$

Das Interaktionsrisiko beträgt:

$$\rho^{\text{seq}}(\sigma) = 1 - (0.98 \cdot 0.95 \cdot 0.97 \cdot 0.92 \cdot 0.99) = 1 - 0.819 \approx 0.181.$$

Obwohl kein Einzelrisiko 0.1 übersteigt, liegt das Gesamtrisiko der Sequenz bei 18,1 % — ein Beleg für Satz 10.

8.6 Komponentendiagramme: Einsatz- und Schnittstellenrisiko

Komponentendiagramme beschreiben die Zerlegung eines Softwaresystems in deploybare Einheiten und deren Schnittstellen. Das Risiko manifestiert sich hier in der Abhängigkeitspropagation zwischen Komponenten.

Definition 21 (Komponentenrisikograph). Sei $\mathcal{G} = (V, E_K)$ ein gerichteter Graph, wobei V die Menge der UML-Komponenten und $E_K \subseteq V \times V$ die Abhängigkeitsbeziehungen (Use-Relationships) bezeichnen. Jeder Kante $(c_i, c_j) \in E_K$ ist ein *Propagationsfaktor* $\pi_{ij} \in [0, 1]$ zugeordnet (`propagationFactor` im `RiskProfile`).

Satz 11 (Risikopropagation in Komponentengraphen). Sei c_0 eine Wurzelkomponente mit Restrisiko ρ_{c_0} und $(c_0, c_1, \dots, c_\ell)$ ein gerichteter Pfad der Länge ℓ in $\mathcal{G}$. Das propagierte Risiko an der Endkomponente c_ℓ ist:

$$\rho^{\text{prop}}(c_\ell) = \rho_{c_\ell} + (1 - \rho_{c_\ell}) \cdot \rho_{c_0} \cdot \prod_{k=0}^{\ell-1} \pi_{c_k c_{k+1}}.$$

Beweis. Das Ereignis „Ausfall von c_ℓ “ tritt ein entweder durch:

- (i) Eigenversagen von c_ℓ mit Wahrscheinlichkeit ρ_{c_ℓ} , oder
- (ii) Versagen durch Propagation aus c_0 : Wahrscheinlichkeit $\rho_{c_0} \cdot \prod_{k=0}^{\ell-1} \pi_{c_k c_{k+1}}$.

Da beide Ereignisse nicht notwendig disjunkt sind, gilt mit der Inklusion-Exklusion-Formel:

$$\begin{aligned}\rho^{\text{prop}}(c_\ell) &= \rho_{c_\ell} + \rho_{c_0} \prod_k \pi_{c_k c_{k+1}} - \rho_{c_\ell} \cdot \rho_{c_0} \prod_k \pi_{c_k c_{k+1}} \\ &= \rho_{c_\ell} + (1 - \rho_{c_\ell}) \cdot \rho_{c_0} \prod_k \pi_{c_k c_{k+1}}.\end{aligned}\quad \square$$

Korollar 6 (Maximalrisiko durch Propagation). Für $\pi_{ij} = 1$ (vollständige Propagation) und $\rho_{c_0} = 1$ gilt:

$$\rho^{\text{prop}}(c_\ell) = 1.$$

Vollständige Abhängigkeit bei maximalem Quellrisiko führt zu absolutem Ausfallrisiko der Zielkomponente — eine formale Begründung für das Prinzip der losen Kopplung (*loose coupling*) in der Softwarearchitektur.

Beweis. Einsetzen in die Formel aus Satz 11: $\rho^{\text{prop}}(c_\ell) = \rho_{c_\ell} + (1 - \rho_{c_\ell}) \cdot 1 \cdot 1 = 1.$ $\square$

8.7 Weitere relevante Diagrammtypen

8.7.1 Zustandsdiagramme

Zustandsdiagramme (*State Machine Diagrams*) modellieren das Verhalten von Objekten als endliche Automaten. Die Risikorelevanz ergibt sich aus der Anzahl erreichbarer Zustände und den Transitionsbedingungen.

Definition 22 (Zustandsraumrisiko). Sei $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ ein endlicher Automat, der das Verhalten einer Klasse C modelliert. Das *Zustandsraumrisiko* ist:

$$\rho^{\text{state}}(\mathcal{A}) = 1 - \frac{|Q_{\text{getestet}}|}{|Q|},$$

wobei $Q_{\text{getestet}} \subseteq Q$ die Menge der durch Testfälle abgedeckten Zustände bezeichnet.

Satz 12 (Zustandsraumrisiko und Testabdeckung). Für jede unvollständige Testmenge gilt $\rho^{\text{state}}(\mathcal{A}) > 0$. Ferner wächst ρ^{state} monoton mit $|Q|$ bei konstanter Testmengengröße.

Beweis. Da $|Q_{\text{getestet}}| \leq |Q_{\text{test}}| < |Q|$ bei unvollständiger Testmenge, gilt $\frac{|Q_{\text{getestet}}|}{|Q|} < 1$, also $\rho^{\text{state}} > 0$. Sei $|Q_{\text{getestet}}|$ konstant und $|Q|$ wachsend. Dann fällt der Bruch $\frac{|Q_{\text{getestet}}|}{|Q|}$ streng monoton, und $\rho^{\text{state}} = 1 - \frac{|Q_{\text{getestet}}|}{|Q|}$ wächst streng monoton. $\square$

8.7.2 Aktivitätsdiagramme

Aktivitätsdiagramme modellieren Arbeitsabläufe und Kontrollflüsse. Das Risiko ergibt sich aus der Komplexität der Verzweigungslogik.

Definition 23 (Kontrollflussrisiko). Sei $G_A = (N, E_A)$ der Kontrollfluss-Graph eines Aktivitätsdiagramms mit $|N|$ Knoten und $|E_A|$ Kanten. Das *Kontrollflussrisiko* basiert auf der zyklomatischen Komplexität [18]:

$$M(G_A) = |E_A| - |N| + 2,$$

und das Kontrollflussrisiko ist:

$$\rho^{\text{ctrl}}(G_A) = 1 - e^{-\eta \cdot M(G_A)}, \quad \eta > 0.$$

Lemma 4 (Kontrollflussrisiko und zyklomatische Komplexität). $\rho^{\text{ctrl}}(G_A)$ ist streng monoton wachsend in $M(G_A)$. Für $M(G_A) \leq 10$ (McCabe-Grenze für wartbare Komplexität [18]) gilt $\rho^{\text{ctrl}}(G_A) < 1 - e^{-10\eta}$.

Beweis. Da $e^{-\eta M}$ streng monoton fallend in $M > 0$ ist, folgt die erste Aussage unmittelbar. Die Schranke folgt durch Einsetzen von $M(G_A) = 10$: $\rho^{\text{ctrl}} = 1 - e^{-10\eta} < 1$. $\square$

8.7.3 Anwendungsfall- und Verteilungsdiagramme

Bemerkung 3 (Anwendungsfalldiagramme). Anwendungsfalldiagramme (*Use Case Diagrams*) geben primär eine Außenperspektive auf das System. Im *RiskProfile* werden Use Cases mit `«riskElement»` annotiert, wobei `uncertainty` die Unklarheit der Anforderungsdefinition und `criticality` die geschäftliche Kritikalität repräsentiert. Ein Use Case mit hoher `uncertainty` korrespondiert direkt mit einer hohen Komponente u_{req} im Unsicherheitsvektor aus Abschnitt 2.

Bemerkung 4 (Verteilungsdiagramme). Verteilungsdiagramme (*Deployment Diagrams*) zeigen die physikalische Verteilung von Softwarekomponenten auf Knoten. Das `deployRisk`-Tagged-Value des Stereotyps `«riskComponent»` modelliert hier das Risiko aus Umgebungsunsicherheit u_{env} — also die Wahrscheinlichkeit, dass der Zielknoten nicht in der Konfiguration vorliegt, für die das System entwickelt wurde.

8.8 Aggregation: Systemweites Risikomodell

Definition 24 (Systemweites Risiko eines UML-Modells). Sei $\mathcal{D} = \{D_1, \dots, D_p\}$ eine Sammlung von UML-Diagrammen (Klassen-, Sequenz-, Komponenten-, Zustands- und Aktivitätsdiagramme), alle annotiert mit Π_R . Das *systemweite Risiko* ist:

$$\rho^{\text{sys}}(\mathcal{D}) = 1 - \prod_{j=1}^p (1 - w_j \cdot \bar{\rho}(D_j)),$$

wobei $w_j \in [0, 1]$ das Gewicht des Diagrammtyps (mit $\sum_j w_j = 1$) und

$$\bar{\rho}(D_j) = \frac{1}{|E_j|} \sum_{e \in E_j} \lambda_e \cdot \rho_e$$

das gewichtete mittlere Elementrisiko des Diagramms D_j bezeichnet, mit E_j als Menge der annotierten Elemente in D_j .

Satz 13 (Risikoaggregationstheorem). Das systemweite Risiko $\rho^{\text{sys}}(\mathcal{D})$ besitzt folgende Eigenschaften:

- (i) $\rho^{\text{sys}}(\mathcal{D}) > 0$ für alle nicht-leeren Diagrammmengen mit $\bar{\rho}(D_j) > 0$.
- (ii) $\rho^{\text{sys}}(\mathcal{D})$ ist streng monoton wachsend in jedem $\bar{\rho}(D_j)$.
- (iii) Es gilt: $\rho^{\text{sys}}(\mathcal{D}) < 1$ genau dann, wenn $\bar{\rho}(D_j) < 1$ für alle j .
- (iv) $\rho^{\text{sys}}(\mathcal{D}) \geq \max_j w_j \cdot \bar{\rho}(D_j)$.

Beweis. (i): Für $\bar{\rho}(D_j) > 0$ gilt $w_j \cdot \bar{\rho}(D_j) > 0$ (da $w_j > 0$), also $(1 - w_j \bar{\rho}(D_j)) < 1$. Das Produkt $\prod_j (1 - w_j \bar{\rho}(D_j)) < 1$, daher $\rho^{\text{sys}} = 1 - \prod_j (\dots) > 0$.

(ii): Sei $\bar{\rho}(D_k)$ wachsend. Dann fällt $(1 - w_k \bar{\rho}(D_k))$ und damit das gesamte Produkt. Also steigt ρ^{sys} .

(iii): $\rho^{\text{sys}} < 1 \Leftrightarrow \prod_j (1 - w_j \bar{\rho}(D_j)) > 0 \Leftrightarrow \forall j: w_j \bar{\rho}(D_j) < 1 \Leftrightarrow \bar{\rho}(D_j) < 1/w_j$. Da $w_j \leq 1$, ist $\bar{\rho}(D_j) < 1$ eine hinreichende Bedingung.

(iv): Es gilt $1 - \prod_j (1 - x_j) \geq x_k$ für alle k , wenn $x_j \in [0, 1]$ (Bernoulli-Ungleichung in umgekehrter Form): $\prod_j (1 - x_j) \leq (1 - x_k)$, also $1 - \prod_j (1 - x_j) \geq x_k$. Mit $x_j = w_j \bar{\rho}(D_j)$ folgt die Behauptung. $\square$

Korollar 7 (Übereinstimmung mit dem Belastbarkeitsmodell). Das systemweite Risiko $\rho^{\text{sys}}(\mathcal{D})$ ist konsistent mit dem Belastbarkeitsindex $B(t, u, r)$ aus Definition 6:

$$B(t, u, r) \leq 1 - \rho^{\text{sys}}(\mathcal{D}) \quad \text{für } t \geq t_0,$$

wobei t_0 der Inbetriebnahmezeitpunkt ist. Das UML-Risikomodell liefert damit eine untere Schranke für den Belastbarkeitsindex ab Produktionsstart.

Beweis. Nach Satz 8 gilt $B \leq 1 - \max_i \lambda_{e_i} \rho_{e_i}$. Nach Satz 13 (iv) gilt $\rho^{\text{sys}} \geq \max_j w_j \bar{\rho}(D_j) \geq \max_i \lambda_{e_i} \rho_{e_i} \cdot \min_j w_j$. Für $\min_j w_j = 1/p$ (Gleichgewichtung) folgt $\rho^{\text{sys}} \geq \frac{1}{p} \max_i \lambda_{e_i} \rho_{e_i}$. Da p endlich und $B \leq 1 - \max_i \lambda_{e_i} \rho_{e_i} \leq 1 - \rho^{\text{sys}}$ im dominanten Fall (ein Diagrammtyp trägt das Maximum), folgt die Behauptung. $\square$

8.9 Relevanz der Diagrammtypen im Vergleich

Satz 14 (Diagrammtypspezifische Risikoschwergewichte). Sei $\mathcal{S}$ ein Softwaresystem der Klasse k (gemäß Definition 11). Die optimalen Diagrammgewichte w_j^* für die Risikoaggregation erfüllen folgende Ungleichungskette:

Klasse	Dominanter Diagrammtyp	Begründung
I	Klassendiagramm	stabile Struktur dominiert
II	Komponentendiagramm	Deployment-Risiko dominiert
III	Sequenzdiagramm	volatile Interaktionen dominieren
IV	alle gleichgewichtig	maximale Gesamtunsicherheit

Beweis. Die Begründung folgt aus der Charakteristik der Vertragsklassen (Definition 11) und den Risikodefinitionen der einzelnen Diagrammtypen:

Klasse I ($\kappa \leq 0.5, \nu \leq 0.5$): Geringe Komplexität und geringe Volatilität bedeuten, dass weder Interaktionen noch Deployment stark schwanken. Das dominante Risiko liegt in der statischen Struktur (Definition 19), also im Klassendiagramm.

Klasse II ($\kappa > 0.5, \nu \leq 0.5$): Hohe technische Komplexität bei stabilen Anforderungen führt zu komplexen Deployment-Topologien (z. B. verteilte Systeme, Echtzeitsysteme). Das Komponentendiagramm mit `deployRisk` und Propagationsfaktoren dominiert.

Klasse III ($\kappa \leq 0.5, \nu > 0.5$): Volatile Anforderungen ändern primär die Interaktionssequenzen zwischen Objekten. Nach Satz 10 akkumuliert sich das Risiko superadditiv in Sequenzdiagrammen; diese dominieren daher.

Klasse IV ($\kappa > 0.5, \nu > 0.5$): Maximale Unsicherheit in allen Dimensionen bedeutet, dass kein einzelner Diagrammtyp das Risiko allein repräsentieren kann. Gleichgewichtung $w_j = 1/p$ maximiert die Robustheit der Risikoabschätzung gemäß Korollar 7. $\square$

9 Rechtskraft von Softwareverträgen unter der Bedingung formaler Risikomodellierung

Die bisher entwickelten Ergebnisse dieser Arbeit — das Belastbarkeitsmodell $B(t, u, r)$, der Satz „Software heißt Risiko“ (Satz 5) und das UML-Risikoprofil RiskProfile (Abschnitt 8) — führen zu einer zentralen rechtlichen Konsequenz, die in diesem Kapitel formal ausgearbeitet wird:

Ein Softwarevertrag kann erst dann als rechtskräftig gelten, wenn die Risikomodellierung gründlich und formal korrekt nachgewiesen wurde.

Diese These ist nicht nur eine praktische Empfehlung, sondern lässt sich aus den formalen Ergebnissen dieser Arbeit logisch ableiten. Im Folgenden wird sie präzisiert, formal begründet und in den rechtlichen Kontext eingebettet.

9.1 Formale Vorbedingung der Vertragsgültigkeit

Definition 25 (Risikonachweis). Ein *Risikonachweis* $\mathcal{N}$ für einen Softwarevertrag $\mathcal{V} = (\mathcal{S}, \Phi, t_0, T_{\max})$ ist ein Tupel:

$$\mathcal{N} = (\Pi_R^*, \rho^{\text{sys*}}, \mathcal{K}^*, \mathcal{P}^*),$$

wobei

- Π_R^* : ein vollständig annotiertes UML-Modell gemäß Definition 18, das alle OCL-Constraints C_1 – C_4 erfüllt,
- $\rho^{\text{sys*}} \in (0, 1)$: das berechnete systemweite Restrisiko gemäß Definition 24, formal hergeleitet aus Π_R^* ,
- $\mathcal{K}^* \in \{\text{I, II, III, IV}\}$: die zugewiesene Vertragsklasse gemäß Definition 11, nachgewiesen durch Messung von κ und ν ,
- $\mathcal{P}^*$: ein formaler Beweis oder eine maschinell geprüfte Verifikation, dass Π_R^* konsistent mit Φ und den Ergebnissen von Abschnitt 8 ist.

Definition 26 (Rechtskräftiger Softwarevertrag). Ein Softwarevertrag $\mathcal{V}$ heißt *rechtskräftig im Sinne dieser Arbeit*, wenn ein gültiger Risikonachweis $\mathcal{N}$ im Sinne von Definition 25 existiert und dieser Bestandteil des Vertragswerks ist:

$$\text{rechtskräftig}(\mathcal{V}) \iff \exists \mathcal{N} \text{ gültig} \wedge \mathcal{N} \subseteq \mathcal{V}.$$

9.2 Notwendigkeit des Risikonachweises: Formale Begründung

Satz 15 (Notwendigkeit des Risikonachweises für Vertragsgültigkeit). Sei $\mathcal{V}$ ein Softwarevertrag ohne gültigen Risikonachweis $\mathcal{N}$. Dann existiert stets eine Situation, in der eine Vertragspartei eine Verpflichtung eingeht, deren Erfüllbarkeit strukturell ungewiss ist — was die informierte Einwilligung (*informed consent*) beider Parteien unmöglich macht.

Beweis. Der Beweis verläuft in drei Schritten.

Schritt 1: Strukturelles Restrisiko ohne Nachweis. Nach Satz 5 gilt für jedes nicht-triviale Softwaresystem $\mathcal{S}$:

$$P(\exists z \in \mathcal{Z}(\mathcal{S}) \setminus \mathcal{T}: \mathcal{S}(z, E) \notin \Phi) > 0.$$

Ohne Risikonachweis $\mathcal{N}$ ist dieses Risiko den Vertragsparteien weder quantifiziert noch kommuniziert. Es existiert also ein positives, den Parteien unbekanntes Restrisiko.

Schritt 2: Fehlende informierte Einwilligung. Ein Vertrag im Sinne des BGB (§ 145 ff.) setzt die Willenserklärung beider Parteien voraus. Eine Willenserklärung ist nur dann inhaltlich bestimmt, wenn der Gegenstand der Verpflichtung hinreichend klar ist [13]. Da ohne $\mathcal{N}$ das systemweite Risiko ρ^{sys} unbekannt ist, kann der Auftraggeber nicht beurteilen, ob er die mit dem Softwaresystem verbundenen Risiken akzeptiert — seine Willenserklärung ist informationsdefizitär.

Schritt 3: Logische Äquivalenz. Sei $I(P)$ die Aussage „Partei P hat informierte Kenntnis der Vertragsverpflichtung“. Ohne $\mathcal{N}$ gilt: $\neg I(\text{Auftraggeber})$, denn ρ^{sys} ist unbekannt. Da ein gültiger Vertrag $I(\text{Auftraggeber}) \wedge I(\text{Auftragnehmer})$ voraussetzt [13], folgt: $\neg \mathcal{N} \Rightarrow \neg \text{rechtskräftig}(\mathcal{V})$. Kontraposition: $\text{rechtskräftig}(\mathcal{V}) \Rightarrow \exists \mathcal{N}$. $\square$

Korollar 8 (Risikonachweis als Vertragsbedingung). Der Risikonachweis $\mathcal{N}$ ist eine *notwendige Bedingung* für die Rechtskraft eines Softwarevertrags. Kein Softwarevertrag, der das strukturelle Restrisiko des zu entwickelnden Systems verschweigt oder nicht formal quantifiziert, kann als vollständig informierter und damit rechtskräftiger Vertrag gelten.

9.3 Hinreichende Bedingungen: Vollständigkeit des Risikonachweises

Definition 27 (Vollständigkeit des Risikonachweises). Ein Risikonachweis $\mathcal{N}$ heißt *vollständig*, wenn er folgende vier Bedingungen erfüllt:

- (i) **Modellvollständigkeit:** Das annotierte UML-Modell Π_R^* umfasst mindestens alle Diagrammtypen, die nach Satz 14 für die Vertragsklasse $\mathcal{K}^*$ dominant sind.
- (ii) **Quantitative Nachvollziehbarkeit:** Alle Tagged Values $(\rho_e, u_e, \lambda_e, \kappa_e)$ in Π_R^* sind entweder durch Messung (statische Analyse, Coverage-Metriken) oder durch begründete Schätzung mit angegebenem Konfidenzintervall belegt.
- (iii) **Aggregationskonsistenz:** Das berechnete systemweite Restrisiko $\rho^{\text{sys}*}$ folgt nachvollziehbar aus den Elementrisiken gemäß der Formel in Definition 24.
- (iv) **Risikoklauseln:** Der Vertrag $\mathcal{V}$ enthält explizite Risikoklauseln $\mathcal{R}_C$ gemäß Definition 25, die $\rho^{\text{sys}*}$ benennen und Haftung sowie Mitigationsmaßnahmen regeln.

Satz 16 (Hinreichende Bedingung für Rechtskraft). Sei $\mathcal{N}$ ein vollständiger Risikonachweis im Sinne von Definition 27. Dann sind beide Vertragsparteien in der Lage, eine informierte Willenserklärung abzugeben, und der Vertrag $\mathcal{V}$ erfüllt die formalen Voraussetzungen der Rechtskraft.

Beweis. Aus Bedingung (i) folgt, dass alle risikodominanten Systemaspekte modelliert sind. Aus Bedingung (ii) folgt, dass die Risikomaßzahlen nicht beliebig angesetzt, sondern empirisch begründet sind — der Auftraggeber kann sie nachprüfen oder durch eigene

Sachverständige prüfen lassen. Aus Bedingung (iii) folgt, dass das Gesamtrisiko $\rho^{\text{sys}*}$ formal aus den Elementen ableitbar ist und keine willkürliche Gesamtangabe darstellt. Aus Bedingung (iv) folgt, dass das Restrisiko nicht nur berechnet, sondern auch vertraglich adressiert ist.

Damit gilt: Beide Parteien kennen Art und Größe des Risikos, seine Verteilung auf Systemkomponenten und die vereinbarte Reaktion im Schadensfall. Die Voraussetzung der informierten Willenserklärung aus Satz 15 ist erfüllt. Die formalen Anforderungen an einen bestimmten und einklagbaren Vertragsinhalt nach §§ 145, 241 BGB sind gegeben. $\square$

9.4 Stufenmodell der Vertragsgültigkeit

Aus den vorstehenden Definitionen und Sätzen ergibt sich ein dreistufiges Modell der Vertragsgültigkeit, das den Zusammenhang zwischen Risikomodellierungstiefe und Rechtskraft strukturiert:

Definition 28 (Stufenmodell der Softwarevertrags-Rechtskraft). Sei $\mathcal{V}$ ein Softwarevertrag. Wir definieren drei Gültigkeitsstufen:

Stufe	Bezeichnung	Bedingung
0	Ungültig	Kein Risikonachweis; strukturelles Restrisiko unbekannt. Nach Satz 15 keine informierte Einwilligung möglich.
1	Bedingt gültig	Risikonachweis $\mathcal{N}$ vorhanden, aber unvollständig (nicht alle Bedingungen von Definition 27 erfüllt). Vertrag ist anfechtbar, da Informationsdefizite bestehen.
2	Vollständig rechtskräftig	Vollständiger Risikonachweis $\mathcal{N}$ gemäß Definition 27. Nach Satz 16 sind alle Voraussetzungen informierter Willenserklärung erfüllt.

Lemma 5 (Monotonie der Gültigkeitsstufen). Sei $\mathcal{N}_1$ ein Risikonachweis, der k der vier Bedingungen aus Definition 27 erfüllt, und $\mathcal{N}_2$ ein Risikonachweis, der $k' > k$ Bedingungen erfüllt. Dann gilt:

$$\text{Gültigkeitsstufe}(\mathcal{N}_2) \geq \text{Gültigkeitsstufe}(\mathcal{N}_1).$$

Mehr erfüllte Nachweisbedingungen erhöhen die Vertragsgültigkeit monoton.

Beweis. Da jede zusätzlich erfüllte Bedingung ein bestehendes Informationsdefizit schließt (Modellvollständigkeit, Quantifizierbarkeit, Aggregationskonsistenz oder vertragliche Verankerung), nähert sich der Risikonachweis monoton der vollständigen informierten Einwilligung. Die Stufenzuweisung in Definition 28 ist eine Treppenfunktion in k , die bei $k = 4$ Stufe 2 erreicht. Da $k' > k$, gilt $\text{Stufe}(k') \geq \text{Stufe}(k)$. $\square$

9.5 Konsequenzen für die Vertragsgestaltung in der Praxis

Bemerkung 5 (Präufpflicht und Due Diligence). Aus Korollar 8 und Satz 16 ergibt sich für die Vertragspraxis eine klare *Due-Diligence-Pflicht*: Auftraggeber und Auftragnehmer sind verpflichtet, vor Vertragsschluss gemeinsam einen vollständigen Risikonachweis $\mathcal{N}$ zu erstellen. Diese Pflicht ist nicht delegierbar — sie erfordert, dass beide Parteien die in Π_R^* enthaltenen Risikomaßzahlen verstehen und akzeptieren. In der Praxis empfiehlt sich die Einbindung eines unabhängigen Sachverständigen, der den Risikonachweis prüft und bestätigt.

Bemerkung 6 (Vertragsklasse und Nachweistiefe). Die erforderliche Nachweistiefe ist abhängig von der Vertragsklasse $\mathcal{K}^*$. Für Klasse I (geringe Komplexität, stabile Anforderungen) kann ein vereinfachter Nachweis genügen; für Klasse IV (hohe Komplexität, volatile Anforderungen) ist nach Satz 14 eine vollständige Annotation aller fünf Diagrammtypen unabdingbar. Dies korrespondiert mit Satz 7: Gerade bei Klasse IV benötigt der Risikonachweis selbst ein interdisziplinäres Team.

Hypothese 2 (Geringere Streitigkeitsrate bei vollständigem Nachweis). Softwareverträge, die einen vollständigen Risikonachweis $\mathcal{N}$ gemäß Definition 27 enthalten, weisen eine statistisch signifikant geringere Rate an Vertragsstreitigkeiten auf als Verträge ohne Risikonachweis, da Erwartungen und Risiken ex ante klar kommuniziert sind.

Bemerkung 7 (Verhältnis zu bestehenden Normen). Das hier entwickelte Stufenmodell ist kompatibel mit und baut auf bestehenden Normen auf. ISO/IEC 25010 definiert Qualitätsmerkmale, die in die Tagged Values des RiskProfile einfließen. ISO/IEC 12207 beschreibt den Softwarelebenszyklusprozess, in den der Risikonachweis als eigene Phase „Risikomodellierung und -nachweis“ integriert werden kann — idealerweise zwischen Anforderungsanalyse und Vertragsschluss. IEC 61508 fordert für sicherheitskritische Systeme bereits eine systematische Risikoanalyse; das vorliegende Stufenmodell verallgemeinert diesen Gedanken auf *alle* Softwareverträge, unabhängig von der Domäne.

10 Zusammenfassung

Diese Arbeit hat ein umfassendes formales Rahmenwerk zur Belastbarkeit von Softwareverträgen, zur quantitativen Risikomodellierung in UML und zur rechtlichen Gültigkeit von Softwareverträgen entwickelt. Die zentralen Ergebnisse lassen sich wie folgt zusammenfassen:

- Softwaresysteme können aufgrund des Halteproblems und der unendlichen Zustandsmenge nicht vollständig formal verifiziert werden (Satz 1, Satz 4).
- Der Belastbarkeitsindex $B(t, u, r)$ fällt monoton mit Zeit, Unsicherheit und Benutzerzahl (Satz 2). Kein Softwarevertrag bleibt dauerhaft belastbar (Korollar 1).
- Das Restrisiko nach Testabschluss ist strukturell positiv und nicht eliminierbar (Korollar 2).
- **Software heißt Risiko**: Absolute Produktionssicherheit ist nicht garantierbar (Satz 5). Dieses Risiko ist normal und muss von allen Beteiligten bewusst eingeplant werden.

- Vier Vertragsklassen (I–IV) ermöglichen eine systematische Risikokategorisierung nach technischer Komplexität und Anforderungsvolatilität (Definition 11, Satz 14).
- Für Klassen III und IV ist ein interdisziplinäres Team zwingend erforderlich (Satz 7).
- Das UML-Risikoprofil **RiskProfile** ermöglicht die quantitative Annotation von Risikomaßzahlen direkt in UML-Struktur- und Verhaltensdiagrammen (Abschnitt 8). Die Risikoannotation ist formal konsistent mit dem Belastbarkeitsmodell $B(t, u, r)$ und erlaubt eine diagrammtypspezifische Risikoaggregation (Sätze 3–13).
- **Rechtskraft setzt Risikonachweis voraus:** Ein Softwarevertrag kann erst dann als rechtskräftig gelten, wenn ein vollständiger Risikonachweis $\mathcal{N}$ gemäß Definition 27 vorliegt und Bestandteil des Vertragswerks ist (Satz 15, Korollar 8, Satz 16). Das dreistufige Gültigkeitsmodell (Definition 28) ordnet Verträge nach ihrer Risikomodellierungstiefe.

11 Ausblick

Die vorliegende Arbeit eröffnet eine Vielzahl von Forschungsrichtungen. Der Ausblick gliedert sich in drei Teile: Zunächst werden Perspektiven aus dem Kapitel zur Rechtskraft von Softwareverträgen (Abschnitt 9) entwickelt, anschließend folgen Perspektiven aus dem UML-Risikokapitel (Abschnitt 8), und abschließend die übergreifenden Forschungsrichtungen zum Belastbarkeitsmodell und seinem weiteren wissenschaftlichen Umfeld.

11.1 Empirische Validierung des Stufenmodells der Vertragsgültigkeit

Das in Definition 28 entwickelte dreistufige Gültigkeitsmodell ist ein formales Konstrukt, das bisher nicht empirisch validiert wurde. Hypothese 2 besagt, dass Verträge mit vollständigem Risikonachweis eine geringere Streitigkeitsrate aufweisen. Eine empirische Untersuchung könnte diese Hypothese anhand von Urteilsdatenbanken (z. B. juris, beck-online, ECLI-Datenbanken) testen: Wie häufig wurde in Softwarevertragsstreitigkeiten das Fehlen einer dokumentierten Risikoabschätzung als Ursache identifiziert? Welche Schadenshöhen sind typisch für Verträge der Klassen I–IV? Diese Daten könnten die Gewichte des Stufenmodells kalibrieren und seine rechtspraktische Relevanz belegen.

11.2 Normative Verankerung des Risikonachweises im Vertragsrecht

Satz 15 und Korollar 8 begründen den Risikonachweis als notwendige Bedingung für informierte Willenserklärungen. Diese Erkenntnis ist bisher nicht in positivem Recht kodifiziert. Zukünftige Arbeiten könnten konkrete Gesetzgebungsvorschläge entwickeln:

- Eine **Ergänzung von § 631 BGB** (Werkvertragsrecht) um eine explizite Risikooffenbarungspflicht für Softwareentwicklungsverträge ab einer definierten Komplexitätsschwelle $\kappa > 0.5$ (Klassen II und IV).

- Eine **EU-Richtlinie zur Softwarevertragstransparenz**, die — analog zur Verbraucherrechterichtlinie 2011/83/EU — vorschreibt, dass dem Auftraggeber vor Vertragsschluss ein verständlicher Risikonachweis übergeben werden muss.
- Die **Integration in den EU-AI-Act** [14]: Für KI-basierte Hochrisikosysteme (Anhang III AI Act) könnte das vollständig annotierte RiskProfile als Teil der technischen Dokumentation nach Anhang IV gefordert werden.

11.3 Schiedsgerichtliche Nutzung des Risikonachweises

Im Streitfall bietet der formale Risikonachweis $\mathcal{N}$ eine sachliche Grundlage für Schiedsverfahren: Da $\rho^{\text{sys}*}$ und die diagrammbasierten Elementrisiken $\omega(e_i)$ numerisch vorliegen, kann ein technischer Sachverständiger prüfen, ob ein aufgetretener Fehler im vertraglich dokumentierten Risikobereich lag oder einen unvorhergesehenen Ausnahmefall darstellt. Dies würde eine objektivere Haftungsabgrenzung ermöglichen: Liegt der Fehler in einem Element mit $\lambda_e \cdot \rho_e > \rho^{\text{sys}*}$, war er im Risikonachweis antizipiert; liegt er außerhalb, ist eine Nachtragsregelung sachgerecht. Zukünftige Arbeiten könnten Musterschiedsklauseln entwickeln, die den Risikonachweis als Beweismittel in Softwarevertragsstreitigkeiten etablieren.

11.4 Werkzeugunterstützung für das RiskProfile

Das in Definition 18 definierte UML-Profil RiskProfile ist derzeit ein rein formales Konstrukt. Ein wesentlicher nächster Schritt ist seine *Implementierung als maschinenlesbare Profilerdatei* im XMI-Format (XML Metadata Interchange), dem OMG-Standard für den Austausch von UML-Modellen [17]. Konkret sind folgende Werkzeugintegrationen denkbar:

- **Eclipse Papyrus**: Das quelloffene UML-Modellierungswerkzeug unterstützt Profilerdateien nativ. Eine RiskProfile-Erweiterung könnte als Eclipse-Plugin verteilt werden und die Tagged-Value-Eingabe für alle sechs Stereotypen aus Definition 18 mit Eingabvalidierung gegenüber den OCL-Constraints C_1 – C_4 ermöglichen.
- **Enterprise Architect (Sparx Systems)**: Weit verbreitetes kommerzielles Werkzeug mit MDG-Technologie (Model Driven Generation), über die RiskProfile als angepasstes Technologiepaket eingebettet werden kann.
- **PlantUML-Erweiterung**: Für die leichtgewichtige, textbasierte UML-Erstellung könnte eine DSL-Erweiterung definiert werden, die Risikoannotationen in der Syntax `@risk(0.82, 0.9)` direkt in Diagrammquelltext ermöglicht und automatisch in Risikosummaries übersetzt.

Eine solche Implementierung würde es Projektteams erstmals erlauben, die in Satz 13 definierte systemweite Risikoaggregation $\rho^{\text{sys}}(\mathcal{D})$ automatisiert und diagrammübergreifend zu berechnen.

11.5 Automatische Risikoberechnung aus Quellcode und Modell

Die Risikomaßzahlen $\omega(e) = (\rho_e, u_e, \lambda_e, \kappa_e)$ aus Definition 17 werden in der vorliegenden Arbeit manuell annotiert. Eine wichtige Forschungsrichtung ist die *automatische Extraktion* dieser Werte aus bestehenden Artefakten:

- **Kopplungskomplexität** κ_e : Fan-in- und Fan-out-Metriken lassen sich aus Java-Bytecode, Python-AST-Analysen oder C++-Include-Graphen automatisch berechnen. Werkzeuge wie SonarQube oder Understand liefern bereits entsprechende Metriken, die in `coupling`-Tagged-Values übertragen werden können.
- **Zustandsraumschätzung für** ρ^{state} : Symbolische Ausführung (z.B. mit KLEE oder Java PathFinder) kann den erreichbaren Zustandsraum $\mathcal{Z}(\mathcal{S})$ approximieren und damit den Testabdeckungsgrad aus Definition 22 quantifizieren.
- **Zyklomatische Komplexität für** ρ^{ctrl} : McCabes Maß $M(G_A)$ aus Definition 23 ist durch statische Codeanalyse direkt berechenbar und kann automatisch in die `residualRisk`-Tagged-Values von Aktivitätsdiagrammelementen übertragen werden.

Ein vollständiger *Risk-Annotation-Pipeline*-Ansatz — von Quellcode und vorhandenen UML-Modellen bis hin zu einem vollständig annotierten `RiskProfile`-Modell — wäre ein bedeutender Beitrag zur modellgetriebenen Qualitätssicherung.

11.6 Validierung des Risikoaggregationstheorems an realen Projekten

Satz 13 (Risikoaggregationstheorem) und Korollar 7 (Übereinstimmung mit dem Belastbarkeitsmodell) stellen formale Resultate dar, die bisher nicht empirisch validiert wurden. Eine wesentliche Forschungsaufgabe besteht darin, anhand realer Projektdaten zu prüfen, ob $\rho^{\text{sys}}(\mathcal{D})$ tatsächlich eine gültige untere Schranke für $1 - B(t, u, r)$ liefert. Hierfür bieten sich Open-Source-Projekte mit langer Versionierungsgeschichte an (z.B. Linux-Kernel, Apache Commons, Eclipse IDE), bei denen sowohl UML-Modelle als auch Bug-Datenbanken und Ausfallhistorien vorliegen. Ein Vergleich zwischen den vorhergesagten Risikowerten aus ρ^{sys} und den tatsächlich beobachteten Fehlerhäufigkeiten würde die Vorhersagequalität des Modells messen und ggf. eine Rekalibrierung der Diagrammgewichte w_j aus Satz 14 motivieren.

11.7 Erweiterung des RiskProfile um zeitliche Dynamik

Das aktuelle `RiskProfile` annotiert statische Risikomaßzahlen, die zum Modellierungszeitpunkt gelten. Da der Belastbarkeitsindex $B(t, u, r)$ aber ein zeitdynamisches Modell ist (Satz 2), liegt es nahe, auch die UML-Risikoannotationen mit Zeitparametern zu versehen. Ein erweitertes Tagged Value `riskDecayRate: Real` könnte für jeden Stereotyp die Rate modellieren, mit der das Restrisiko des Elements mit der Zeit wächst:

$$\rho_e(t) = 1 - e^{-\alpha_e \cdot t}, \quad \alpha_e > 0.$$

Damit wäre es möglich, zeitliche Risikoentwicklungen direkt im UML-Modell sichtbar zu machen — etwa durch eine farbkodierte Animation in einem UML-Modellierungswerkzeug,

die zeigt, wie einzelne Klassen, Komponenten und Nachrichten mit wachsender Systemlaufzeit riskanter werden.

11.8 Integration des RiskProfile in agile Entwicklungsprozesse

Agile Methoden wie Scrum oder Kanban planen in kurzen Iterationen und reagieren flexibel auf veränderte Anforderungen — genau jene Situation, die Klasse III und IV in Definition 11 beschreibt. Das RiskProfile bietet hier einen formalen Ankerpunkt: Die Risikomaßzahlen können als *Definition of Ready* und *Definition of Done* in User Stories integriert werden. Konkret könnte ein Sprint-Abschluss verlangen, dass kein Element mit $\lambda_e \cdot \rho_e > 0.5$ ohne Risikobehandlungsmaßnahme in den Produktionsbranch übernommen wird. Dieses Kriterium ist direkt aus Satz 8 ableitbar und würde die formalen Ergebnisse dieser Arbeit in die tägliche Entwicklungspraxis übertragen.

11.9 RiskProfile und modellbasierte Sicherheitsnachweise

In sicherheitskritischen Domänen (Automotive nach ISO 26262, Avionik nach DO-178C, Medizintechnik nach IEC 62304) ist der Nachweis der funktionalen Sicherheit Pflicht. Diese Normen fordern u. a. eine systematische Risikoanalyse, die bislang in getrennten Artefakten (FMEA, FTA, HAZOP) geführt wird. Das RiskProfile könnte hier als *Brücke zwischen UML-Systemmodell und Sicherheitsnachweis* dienen: Indem die Risikomaßzahlen $\omega(e)$ direkt im Systemmodell annotiert sind, entfällt die manuelle Übertragung zwischen Modell und Sicherheitsartefakt. Insbesondere könnte das Propagationsmodell aus Satz 11 als formale Grundlage für eine modellbasierte FMEA-Automatisierung dienen: Aus dem Komponentenrisikograph $\mathcal{G} = (V, E_K)$ lassen sich Fehlerbaumstrukturen algorithmisch ableiten, was den Nachweisaufwand erheblich reduzieren würde.

11.10 Diagrammtypgewichte und maschinelles Lernen

Satz 14 legt eine qualitative Begründung für die optimalen Diagrammgewichte w_j^* je Vertragsklasse vor. Eine datengetriebene Alternative besteht darin, diese Gewichte durch maschinelles Lernen aus historischen Projektdaten zu schätzen. Gegeben eine Datenbank von abgeschlossenen Projekten mit bekannter Vertragsklasse, gemessenen Fehlerhäufigkeiten pro Diagrammtyp und beobachteten Belastbarkeitsverläufen $B(t)$, könnten Regressionsmodelle oder neuronale Netze die Parameter w_j so kalibrieren, dass die Vorhersage von ρ^{sys} den beobachteten Fehlerquoten am besten entspricht. Ansätze wie *Survival Analysis* mit zeitabhängigen Kovariaten [15] bieten hierfür einen natürlichen statistischen Rahmen.

11.11 Quantitative Kalibrierung des Belastbarkeitsmodells

Das in Definition 6 eingeführte Modell $B(t, u, r) = e^{-\delta(u, r)t}$ enthält die Parameter δ_0, α, β , die in dieser Arbeit als qualitativ gegeben behandelt wurden. Eine naheliegende Forschungsaufgabe ist die *empirische Kalibrierung* dieser Parameter anhand realer Projektdaten. Hierfür können Datenbanken wie der ISBSG-Datensatz [11], die Chaos-Reports der Standish-Group [10] oder interne Projektarchive von Softwareunternehmen verwendet werden. Regressionsanalysen und maschinelles Lernen können dabei helfen, domänenspezifische Parameterwerte zu schätzen. Die in Abschnitt 8 hergeleiteten

Risikomaßzahlen ρ^{struct} , ρ^{seq} und ρ^{prop} bieten dabei zusätzliche erklärende Variablen, die die Kalibrierungsqualität gegenüber rein zeitbasierten Modellen verbessern könnten.

11.12 Stochastische Erweiterung

Das aktuelle Modell ist deterministisch. Eine wichtige Erweiterung besteht darin, $B(t, u, r)$ als stochastischen Prozess zu formulieren. Denkbar ist ein *geometrischer Brownscher Prozess*:

$$dB = -\delta(u, r) B dt + \sigma B dW_t,$$

wobei W_t ein Wiener-Prozess und $\sigma > 0$ die Volatilität des Projekts ist. Dieses Modell würde es ermöglichen, Konfidenzintervalle für $B(t)$ zu berechnen und das Risiko eines plötzlichen Vertragsversagens zu quantifizieren. Die stochastischen Risikomaßzahlen aus dem **RiskProfile** — insbesondere **failProb** und **propagationFactor** — könnten dabei als diagrammbasierte Schätzer für σ dienen und die stochastische Modellierung empirisch verankern.

11.13 Spieltheoretische Modellierung

Softwareverträge involvieren mindestens zwei strategisch handelnde Akteure: Auftraggeber und Auftragnehmer. Beide haben asymmetrische Information: Der Auftragnehmer kennt die technischen Risiken besser; der Auftraggeber kennt seine geschäftlichen Anforderungen besser. Diese Situation lässt sich als *Bayesianisches Spiel* modellieren [8], in dem Gleichgewichte der Informationsweitergabe (*cheap talk, signaling*) untersucht werden. Wann ist es für den Auftragnehmer rational, Risiken offenzulegen? Das **RiskProfile** könnte hier als *Signaling-Instrument* dienen: Ein Auftragnehmer, der sein UML-Modell vollständig mit **RiskProfile** annotiert und dem Auftraggeber zur Verfügung stellt, sendet ein glaubwürdiges Signal über seine Risikobereitschaft zur Transparenz — was nach Mechanism-Design-Argumenten [8] vertrauensbildend und vertragsstärkend wirkt.

11.14 Formale Methoden und Vertragsprüfung

Die Ergebnisse dieser Arbeit legen nahe, dass formale Methoden — wie Model Checking [6], abstrakte Interpretation [5] oder Hoare-Logik [4] — systematischer in den Vertragslebenszyklus eingebettet werden sollten. Konkret könnten formale Spezifikationssprachen wie TLA+ [7], Alloy oder Event-B dazu verwendet werden, kritische Teile der Vertragsspezifikation Φ maschinenverifikabel zu machen. Eine besonders vielversprechende Verknüpfung besteht zwischen dem **RiskProfile** und Alloy: Die OCL-Constraints C_1 – C_4 aus Definition 18 könnten direkt in Alloy-Prädikate übersetzt werden, sodass die strukturelle Konsistenz des annotierten Modells automatisch geprüft werden kann. Dies würde zwar nicht das Restrisiko aus Korollar 2 eliminieren, aber für die spezifizierten Teile eine nachweislich höhere Belastbarkeit erreichen.

11.15 Rechtsinformatik: Softwareverträge und Produkthaftung

Die Frage der Haftung für Softwarefehler im Produktionsbetrieb ist in der Rechtspraxis nach wie vor ungeklärt [13]. Satz 5 liefert eine formale Grundlage für die These, dass absolute Gewährleistung für Software prinzipiell nicht zugesichert werden kann. Das

RiskProfile ergänzt dies um eine diagrammbasierte Nachweisführung: Ein vollständig annotiertes UML-Modell, das $\rho^{\text{sys}}(\mathcal{D}) > 0$ formal dokumentiert, könnte als prozessualer Nachweis im Sinne einer *Due-Diligence-Verpflichtung* des Auftragnehmers dienen. Dies hat direkte Implikationen für:

- die Formulierung von Gewährleistungs- und Haftungsklauseln in AGB und Werkverträgen nach §§ 631 ff. BGB,
- die Auslegung der EU-Produkthaftungsrichtlinie im Kontext von Softwaresystemen,
- die Anforderungen an Risikobewertungen im Rahmen des EU-AI-Acts [14] für KI-basierte Softwaresysteme, wo das RiskProfile als Bestandteil der technischen Dokumentation nach Anhang IV des AI Acts dienen könnte.

11.16 Einbettung in Normen und Standards

Bestehende Normen wie ISO/IEC 25010 (Softwarequalitätsmodell) [12], ISO/IEC 12207 (Softwarelebenszyklus) und IEC 61508 (funktionale Sicherheit) adressieren Risiken auf prozessualer Ebene, ohne das strukturelle Restrisiko formal zu begründen. Eine Erweiterung dieser Normen um ein quantitatives Belastbarkeitsmodell wäre wünschenswert. Insbesondere könnte die Vertragsklassen-Taxonomie aus Definition 11 als Grundlage für eine neue Norm zur *Risikokommunikation in Softwareverträgen* dienen. Das RiskProfile aus Definition 18 wäre dabei ein natürlicher Kandidat für einen normativen Anhang, der vorschreibt, welche UML-Diagramme für welche Vertragsklasse mit Risikoannotationen versehen sein müssen.

11.17 Maschinelles Lernen zur Belastbarkeitsvorhersage

Moderne KI-Methoden bieten die Möglichkeit, den Belastbarkeitsindex $B(t, u, r)$ datengetrieben zu schätzen. Ansätze wie *Survival Analysis* (z. B. Cox-Proportional-Hazards-Modelle [15]) oder *Recurrent Neural Networks* können auf historische Projektdaten trainiert werden, um frühzeitig Warnsignale für einen drohenden Belastbarkeitsverfall zu erkennen. Die in Abschnitt 8 eingeführten diagrammtypspezifischen Risikomaßzahlen — ρ^{struct} , ρ^{seq} , ρ^{prop} , ρ^{state} und ρ^{ctrl} — stellen dabei einen reichhaltigen Feature-Vektor bereit, der als Eingabe für maschinelle Lernmodelle dienen kann. Ein solches Modell würde ein *prädiktives Vertragsmonitoring-System* ermöglichen, das Projektmanager auf Basis des UML-Modells rechtzeitig auf kritische Risikoschwellen hinweist.

11.18 Psychologie der Risikowahrnehmung

Satz 5 liefert die formale Begründung, dass Softwarerisiken normal sind. Dennoch zeigt die Praxis, dass Risikooffenbarung in Projekten häufig unterbleibt — sei es aus Reputationsangst, kommerziellem Druck oder kognitiven Verzerrungen wie dem *Optimism Bias* [16]. Das RiskProfile könnte hier als Objektivierungswerkzeug wirken: Wenn die Risikomaßzahlen automatisch aus dem Modell berechnet werden (vgl. den Ausblick zur automatischen Risikoberechnung), sind sie nicht mehr das Ergebnis einer subjektiven Einschätzung, sondern folgen formal aus der Modellstruktur. Zukünftige Arbeiten könnten untersuchen, ob eine solche Objektivierung die Risikoberichterstattung in Teams verbessert und wie Prozesse gestaltet werden können, die eine offene Risikokultur fördern.

11.19 Einfluss künstlicher Intelligenz auf Vertragsbelastbarkeit

KI-gestützte Softwareentwicklung (z. B. durch Large Language Models) verändert die Dynamik von $u_{\text{tech}}(t)$ erheblich: Einerseits können KI-Tools die technische Unsicherheit in der Entwicklungsphase reduzieren, andererseits führen sie neue, schwer verifikable Unsicherheitsquellen ein (Halluzinationen, nicht-deterministische Ausgaben, mangelnde Erklärbarkeit). Für das RiskProfile ergibt sich daraus eine besondere Herausforderung: KI-generierter Code besitzt keinen expliziten UML-Entwurf, aus dem die Risikomaßzahlen extrahiert werden könnten. Es ist daher eine offene Forschungsfrage, wie das RiskProfile auf KI-generierte Codebasis angewendet werden kann — etwa durch *reverse-engineerte* UML-Modelle aus KI-Code — und welche neuen Stereotypen (z. B. `«riskAIComponent»`) für KI-Komponenten in das Profil aufgenommen werden sollten.

11.20 Internationale und interdisziplinäre Vertragsstandards

Die unterschiedlichen Rechtssysteme weltweit (Common Law vs. kontinentales Recht) behandeln Softwareverträge und Produkthaftung verschieden. Eine vergleichende Analyse — ergänzt durch das formale Modell dieser Arbeit und das RiskProfile — könnte den Weg für internationale Mindeststandards zur Risikooffenbarung in Softwareverträgen ebnen. Das RiskProfile bietet dabei den Vorteil, dass es sprachunabhängig ist: Die formalen Risikomaßzahlen $\omega(e)$ und das systemweite Risiko $\rho^{\text{sys}}(\mathcal{D})$ sind unabhängig vom Rechtssystem interpretierbar und könnten daher als gemeinsame technische Grundlage für internationale Vertragsstandards dienen, ähnlich wie es im Bereich der Finanzinstrumente durch MiFID II oder im Medizinbereich durch MDR 2017/745 bereits etabliert ist.

References

- [1] Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42), 230–265.
- [2] Dijkstra, E. W. (1972). The Humble Programmer. *Communications of the ACM*, 15(10), 859–866.
- [3] Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Softwareentwicklung*. Addison-Wesley.
- [4] Hoare, C. A. R. (1969). An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10), 576–580.
- [5] Cousot, P. & Cousot, R. (1977). Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs. *Proceedings of the 4th ACM POPL*, 238–252.
- [6] Clarke, E. M., Grumberg, O., & Peled, D. A. (1999). *Model Checking*. MIT Press.
- [7] Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Softwareentwicklung*. Addison-Wesley.
- [8] Harsanyi, J. C. (1967). Games with Incomplete Information Played by Bayesian Players. *Management Science*, 14(3), 159–182.

- [9] IEEE Computer Society (2014). *Guide to the Softwareentwicklung Body of Knowledge (SWEBOK), Version 3.0*. IEEE Press.
- [10] Standish Group (2020). *CHAOS Report 2020: Beyond Infinity*. Standish Group International.
- [11] ISBSG (2020). *ISBSG Benchmark Repository*. International Software Benchmarking Standards Group.
- [12] ISO/IEC 25010:2011. *Systems and Softwareentwicklung – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*. International Organization for Standardization.
- [13] Kilian, W. (2017). *Computerrechtshandbuch: Informationstechnologie in der Rechts- und Wirtschaftspraxis*, 35. Erg.-Lfg. C. H. Beck.
- [14] Europäisches Parlament und Rat (2024). *Verordnung (EU) 2024/1689 des Europäischen Parlaments und des Rates über künstliche Intelligenz (AI Act)*. Amtsblatt der Europäischen Union.
- [15] Cox, D. R. (1972). Regression Models and Life-Tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187–202.
- [16] Weinstein, N. D. (1980). Unrealistic Optimism About Future Life Events. *Journal of Personality and Social Psychology*, 39(5), 806–820.
- [17] Object Management Group (2017). *Unified Modeling Language Specification, Version 2.5.1*. OMG Document Number: formal/2017-12-05. <https://www.omg.org/spec/UML/2.5.1>
- [18] McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, 2(4), 308–320.
- [19] Parnas, D. L. (1972). On the Criteria to Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12), 1053–1058.
- [20] Sommerville, I. (2011). *Softwareentwicklung*, 9th ed. Addison-Wesley.
- [21] Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.